Estruturas de Dados

Heap Binária



2022/2023

Heap Binária

- Suporta a inserção de elementos e remoção do menor em tempo logarítmico (pior caso)
- Usada para implementar *priority queues*.



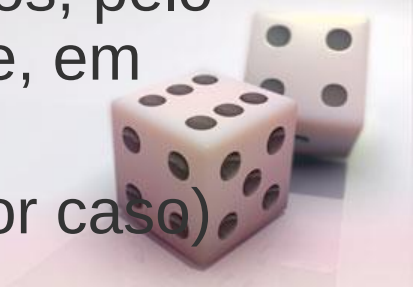
Introdução

- Como implementar um priority queue?
  - **Lista?:** inserção O(1), pesquisa de mínimo O(N)
  - **Lista ordenada?:** inserção O(N), pesquisa de mínimo O(1)
  - **Árvore Binária Equilibrada?** Inserção O(log N), pesquisa mínimo O(log N)
    - No entanto, para dados pouco uniformes, serão necessárias operações de reequilibrio.
    - Não é uma má opção, mas talvez seja possível fazer melhor, porque não é necessário suportar exactamente as mesmas operações eficazmente:
      - basta que a inserção e remoção do mínimo sejam eficazes.



Introdução

- Aquilo que queremos é um compromisso entre a simplicidade de uma *Fila* e as funcionalidade de uma *Árvore*.
- Este compromisso é obtido pela *heap* binária.
  - Pode ser implementada com um *array*.
  - Suporta inserção e remoção do mínimo em O(log N) (pior caso, para inserção corresponde a inserção do mínimo)
  - No entanto, demonstra-se que em termos *médios* a inserção requer apenas 2.6 comparações, independentemente do número de elementos, pelo que a inserção demora um tempo constante, em média.
  - pesquisa do mínimo em tempo em O(1) (pior caso)



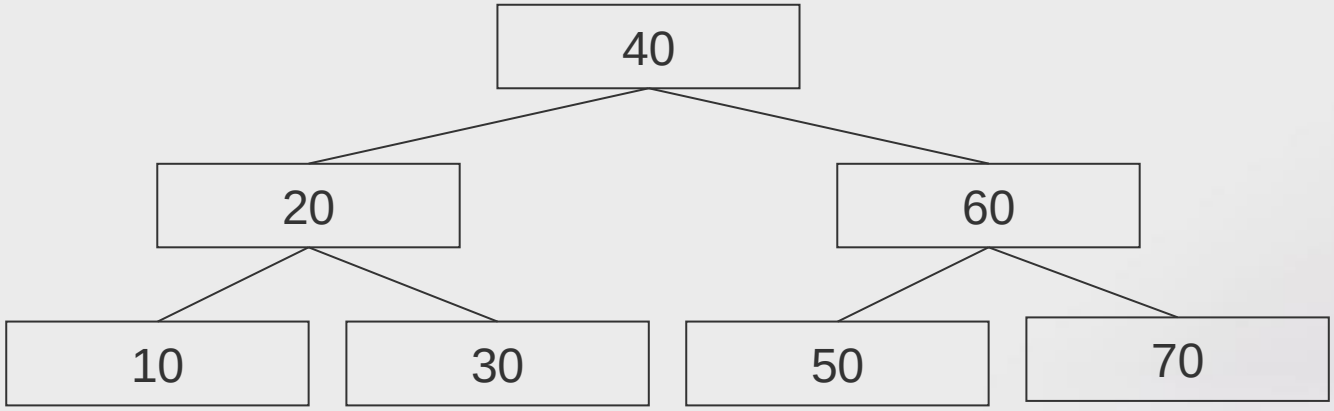
Introdução

- **Heap Binária**
- Tal como nas árvores binárias equilibradas, devem ser observadas algumas propriedades
  - Propriedade de Ordem
  - Propriedade de Estrutura



Propriedade de Estrutura

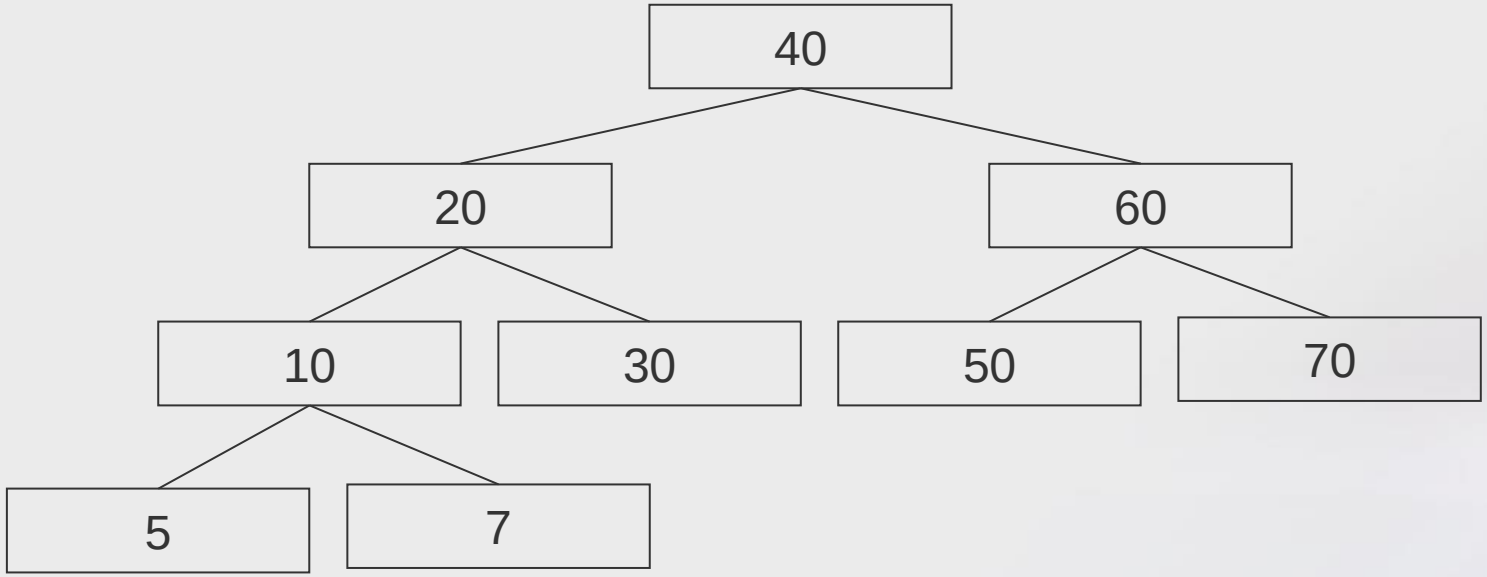
- As árvores são um bom ponto de partida para obter desempenho O(log N):
  - *Árvore Binária Completa:* Árvore completamente preenchida (excepto, eventualmente, último nível)





Propriedade de Estrutura

- As árvores são um bom ponto de partida para obter desempenho O(log N):
  - *Árvore Binária Completa:* Árvore completamente preenchida (excepto, eventualmente, último nível)





Estrutura

- As ABCs são óptimas:
  - Uma árvore de altura H tem entre 2^H e 2^{H+1}-1 nodos.
  - O comportamento dos algoritmos que percorrem a árvore entre a raiz e uma folha são log(N) no pior caso.



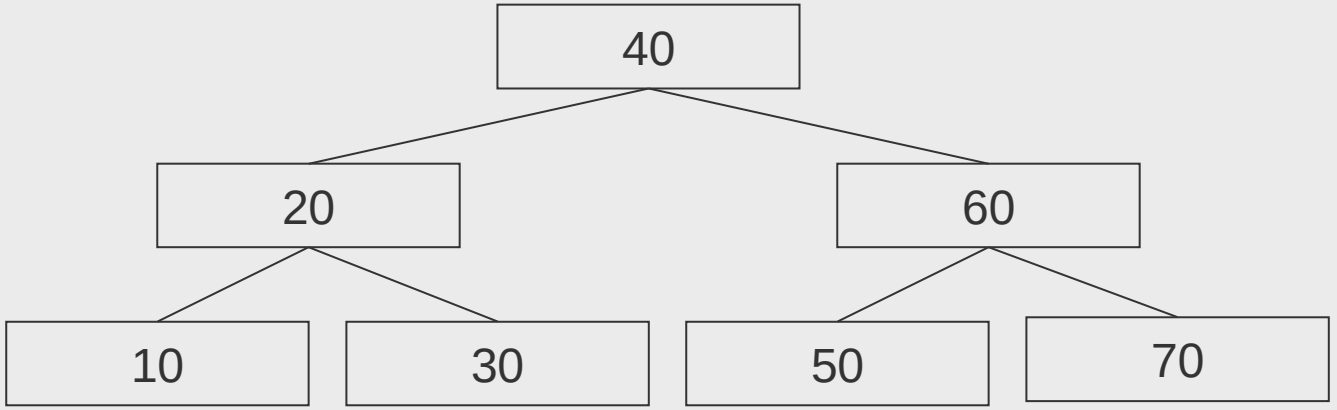
Estrutura

- Outra propriedade interessante das ABCs é que podem ser representadas sem utilizar directamente a estrutura em árvore:
  - Os nodos podem ser guardados num *array* em ordem de nível.



Estrutura

- Exemplo: Representação implícita de árvore.



numeroNodos=7

	40	20	60	10	30	50	70			…

A posição 0 é deliberadamente deixada vazia…



Estrutura

- Exemplo:

numeroNodos=7

	40	20	60	10	30	50	70		

…

•Qual é a posição do descendente esquerdo do nodo na posição ***i*** ?



Estrutura

- Exemplo:

numeroNodos=7

	40	20	60	10	30	50	70		

…

•Qual é a posição do descendente esquerdo do nodo na posição ***i*** ? **2i**
•Qual é a posição do descendente direito do nodo na posição ***i*** ?



Estrutura

- Exemplo:

numeroNodos=7

	40	20	60	10	30	50	70		

…

•Qual é a posição do descendente esquerdo do nodo na posição ***i*** ? ***2i***
•Qual é a posição do descendente direito do nodo na posição ***i*** ? ***2i+1***

A posição 0 é deixada vazia de forma a que este mecanismo de acesso funcione para todas as posições



Propriedade de Ordem-Heap

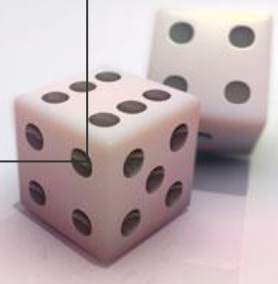
- Queremos procurar eficientemente o menor elemento.
  - Qual o melhor sítio para o colocar?



Propriedade de Ordem-Heap

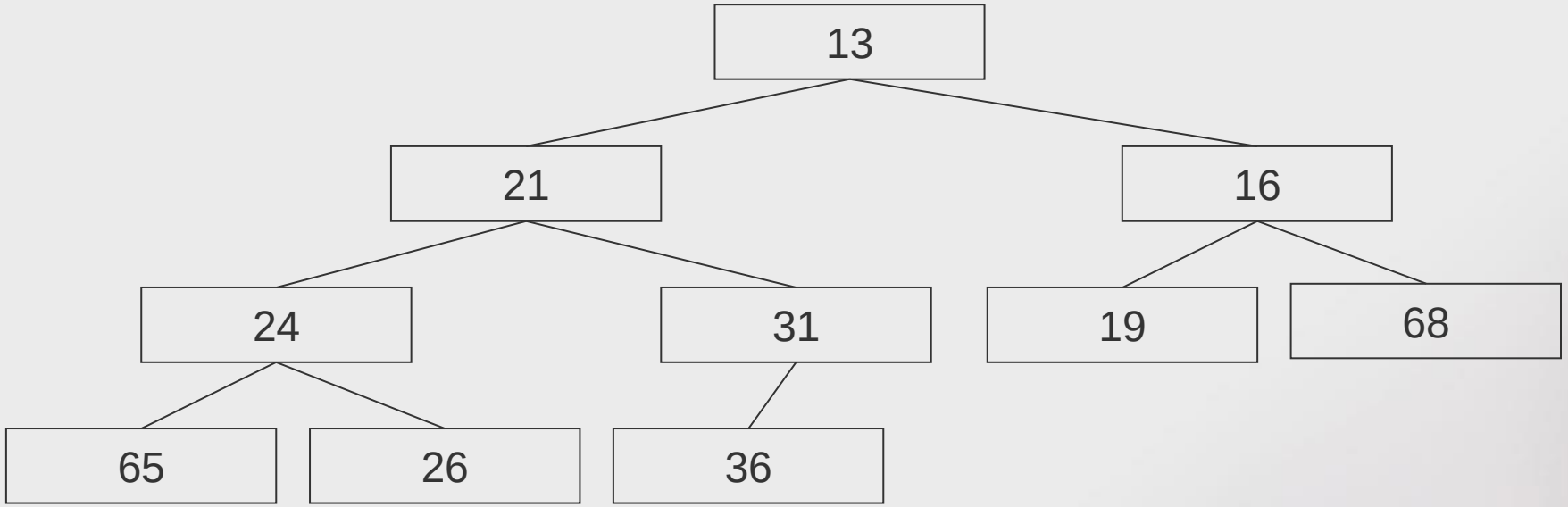
- Queremos procurar eficientemente o menor elemento.
  - Qual o melhor sítio para o colocar?
    - **Na raiz, naturalmente!**
  - Se cada sub-árvore respeitar esta propriedade então concluímos que a propriedade de ordem nas *heaps binárias* pode ser descrita como

Numa *heap binária*, para qualquer nodo X com ascendente P, verifica-se que a chave de P é menor ou igual à chave de X.



Heap Binária

- Exemplo: Heap Binária





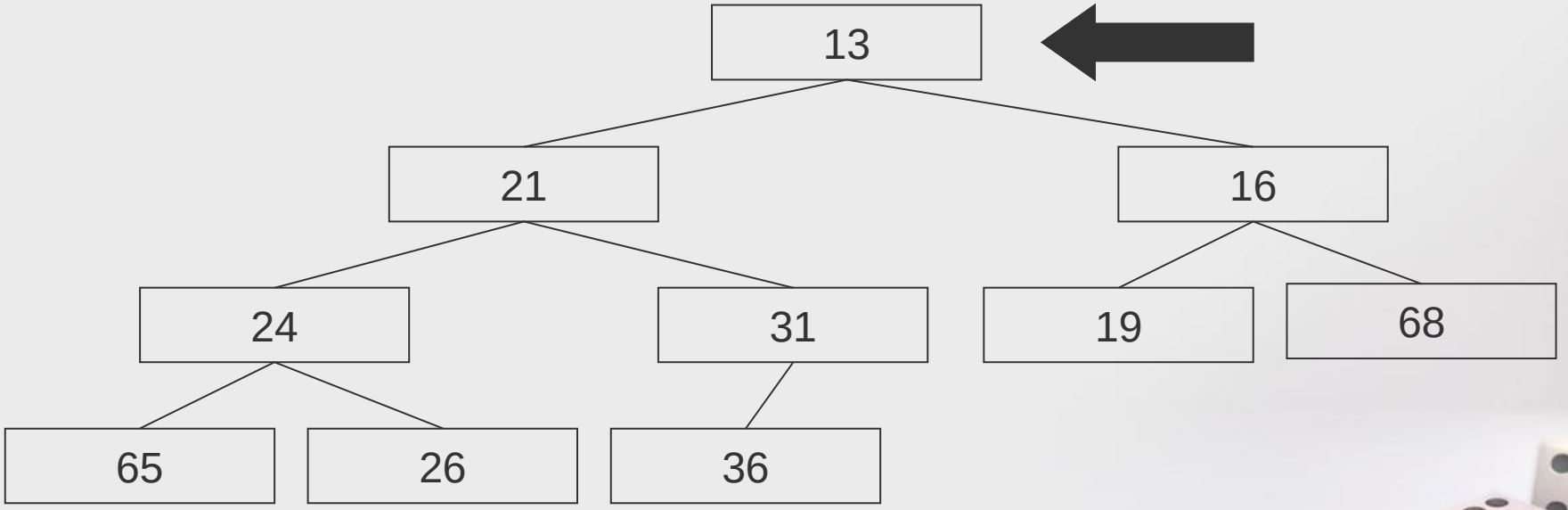
Heap Binária

- Numa *Heap Binária* devem então verificar-se as seguintes propriedadas:
  - *Propriedade de Estrutura*: O conteúdo deve estar organizado numa ABC.
  - *Propriedade de Ordem* (Heap): O ascendente deve sempre ser menor ou igual aos seus descendentes.



Heap Binária

- **Operações:**
  - **Procura de Mínimo?**
    - **Basta analisar o valor na raiz – O(1)**





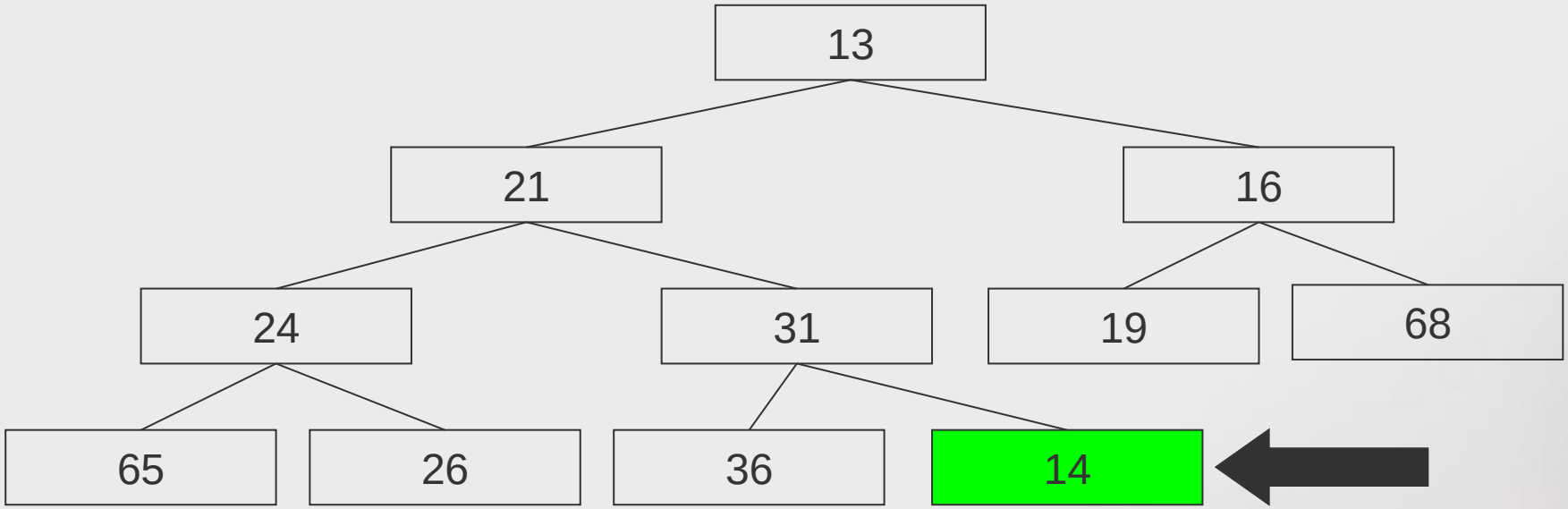
Heap Binária

- **Operações:**
  - **Inserção?**
    - **Coloca-se o valor na próxima posição livre.**
    - **O novo elemento é propagado para cima até que a propriedade de ordem de *heap* seja verificada.**
      - **chama-se a esta operação *Percolar para cima*.**



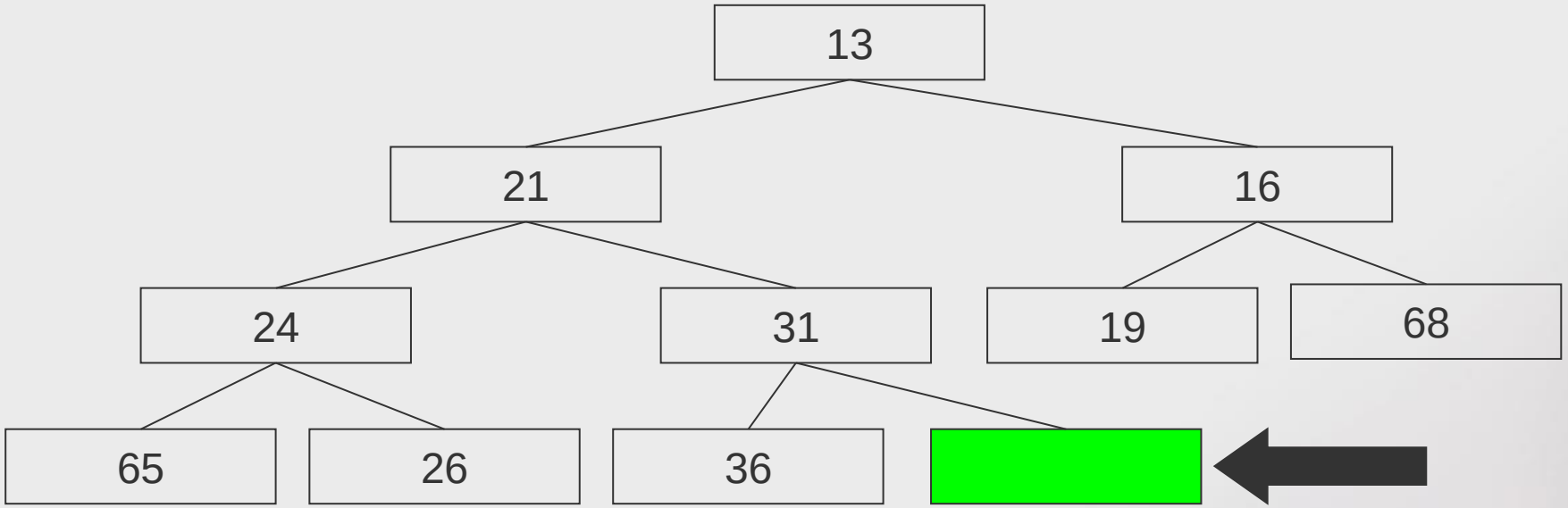
Heap Binária

- Operações: Insere (14)



Heap Binária

- Operações: Insere (14)

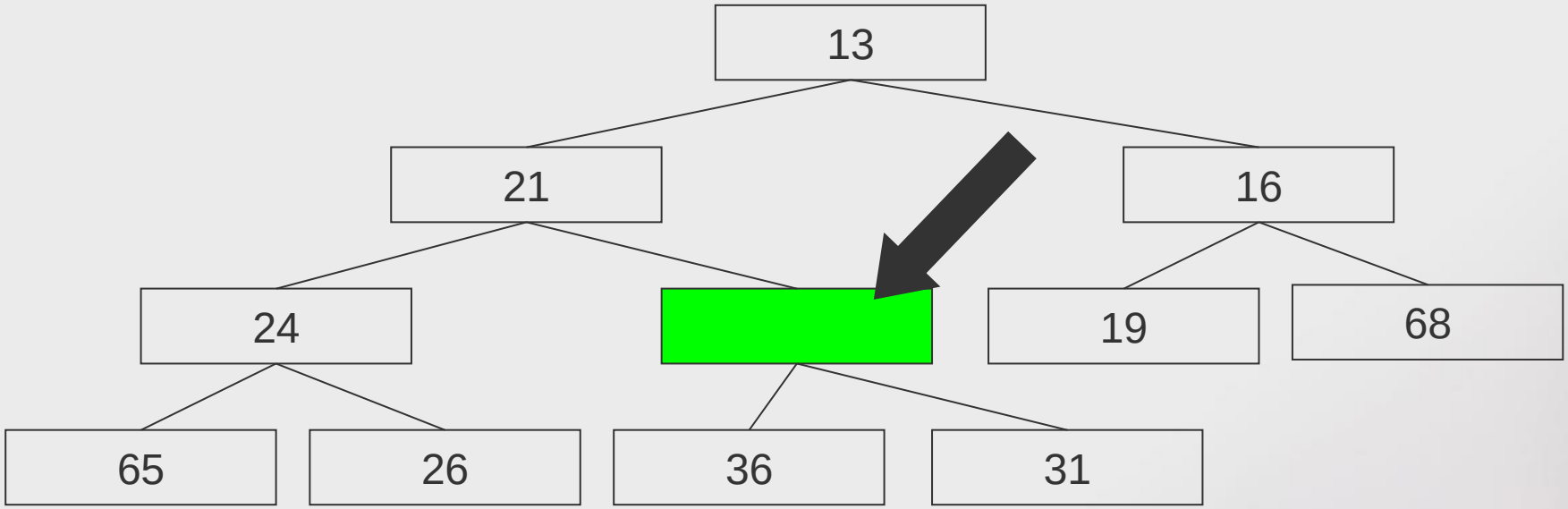


Esta é a próxima posição livre, mas a inserção neste ponto iria violar a propriedade de ordem, pelo que puxamos o ascendente para baixo.



Heap Binária

- Operações: Insere (14)

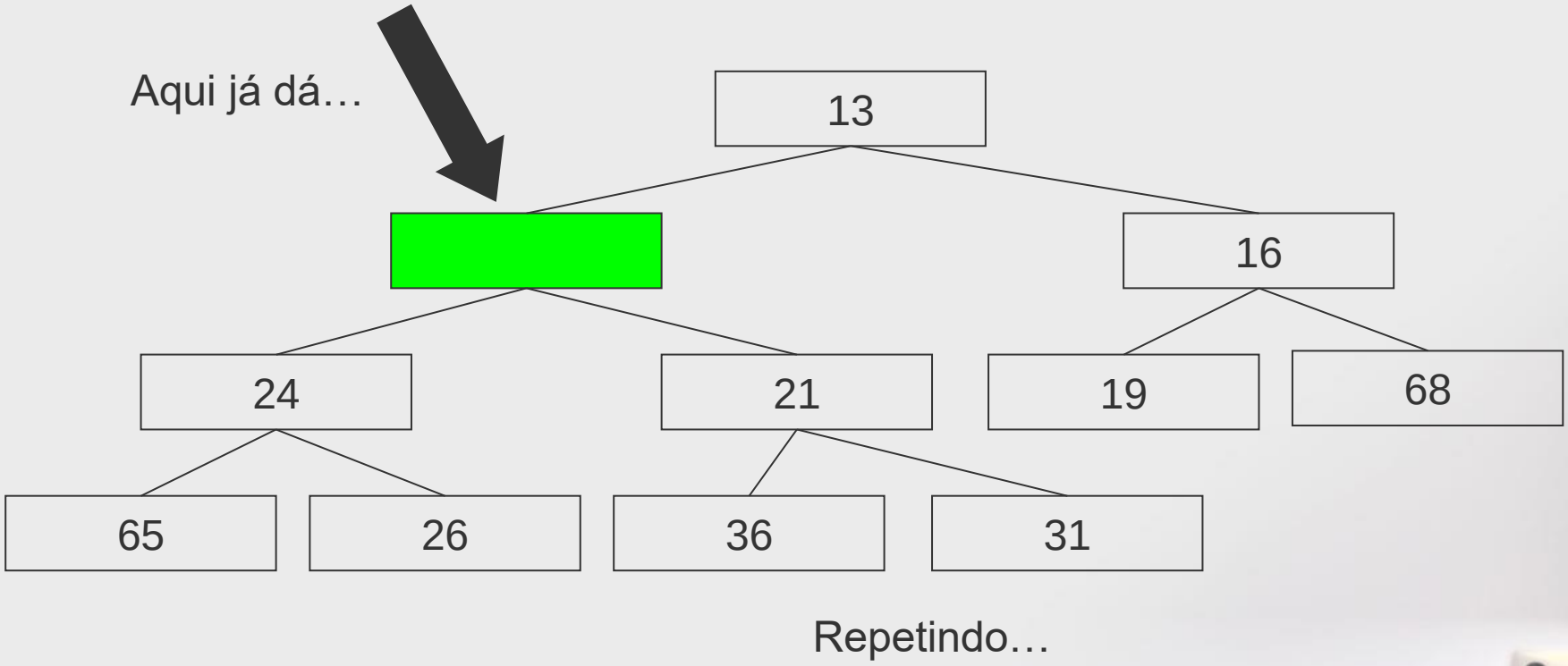


A inserção iria violar a propriedade de ordem, pelo que puxamos o ascendente para baixo.



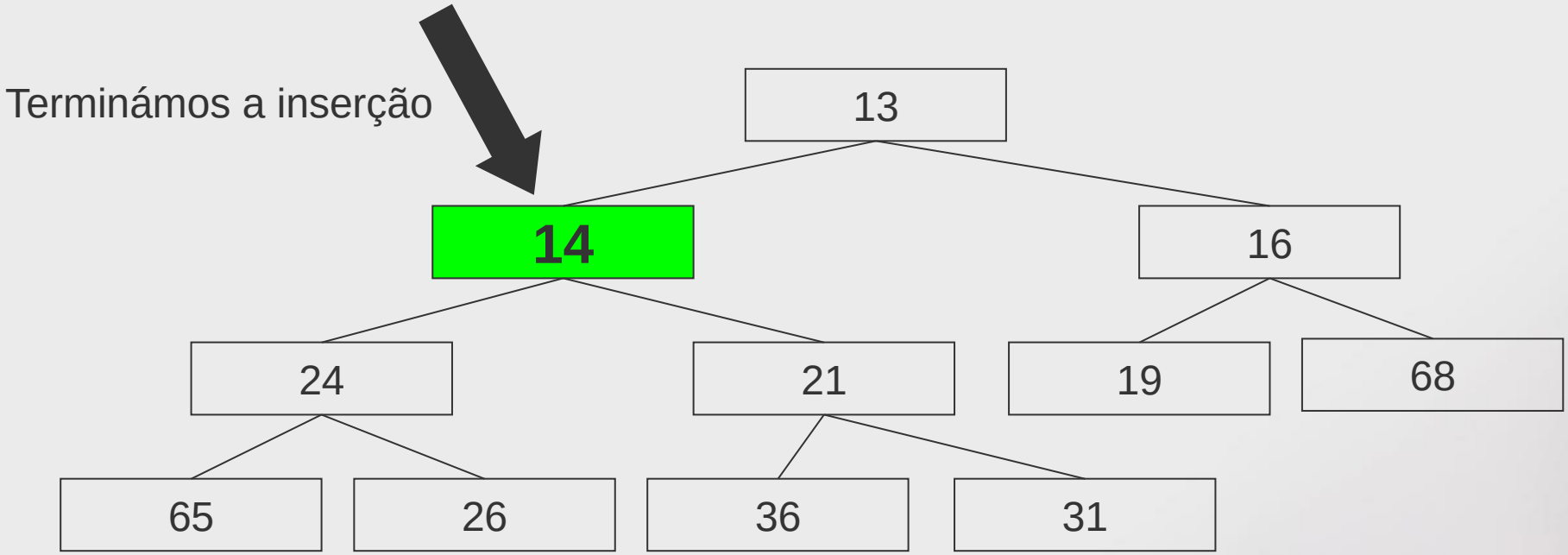
Heap Binária

- Operações: Insere (14)



Heap Binária

- Operações: Insere (14)



Complexidade O(log N) máxima: inserção do novo mínimo faz percurso completo até raiz
Complexidade O(1) média: É necessário, em média, subir 1.6 níveis.



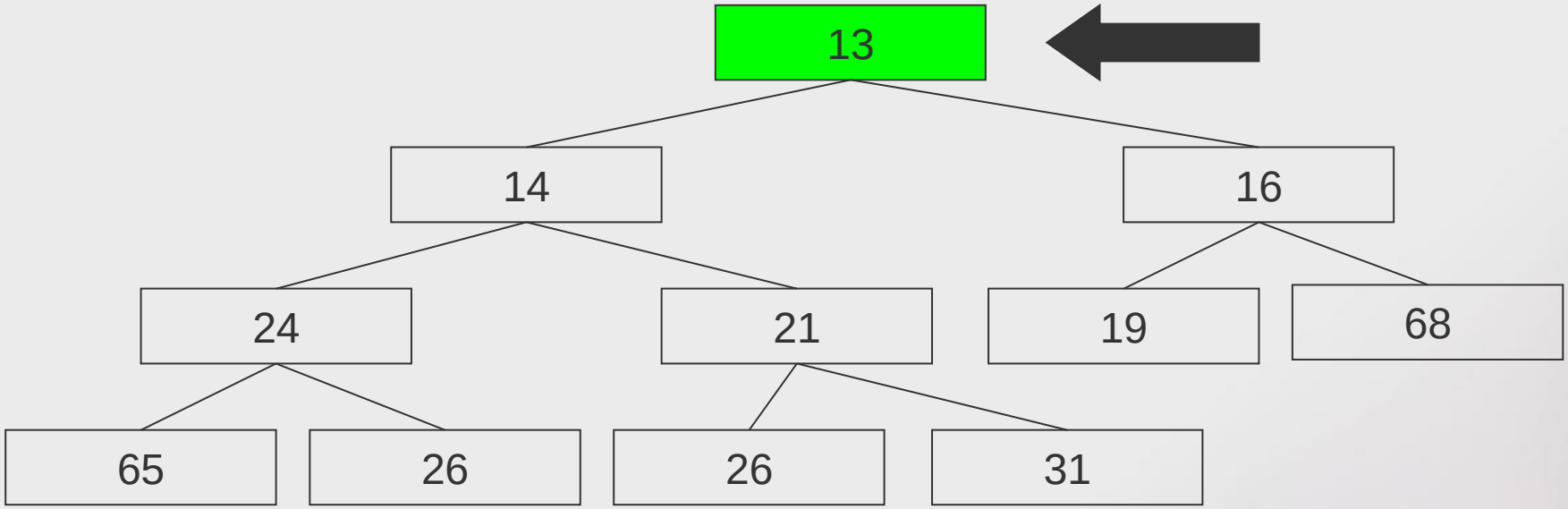
Heap Binária

- **Operações:**
  - **Remoção?**
    - **Retira-se a raiz, o que deixa um "lugar vago" no topo da árvore...**
    - **O último nodo do último nível deve ser reposicionado.**
    - **Enquanto não é possível colocar o último no "lugar vago", este é empurrado para baixo, trocando de posição com o menor descendente…**
      - **Chama-se a esta operação *Percolar para baixo*.**



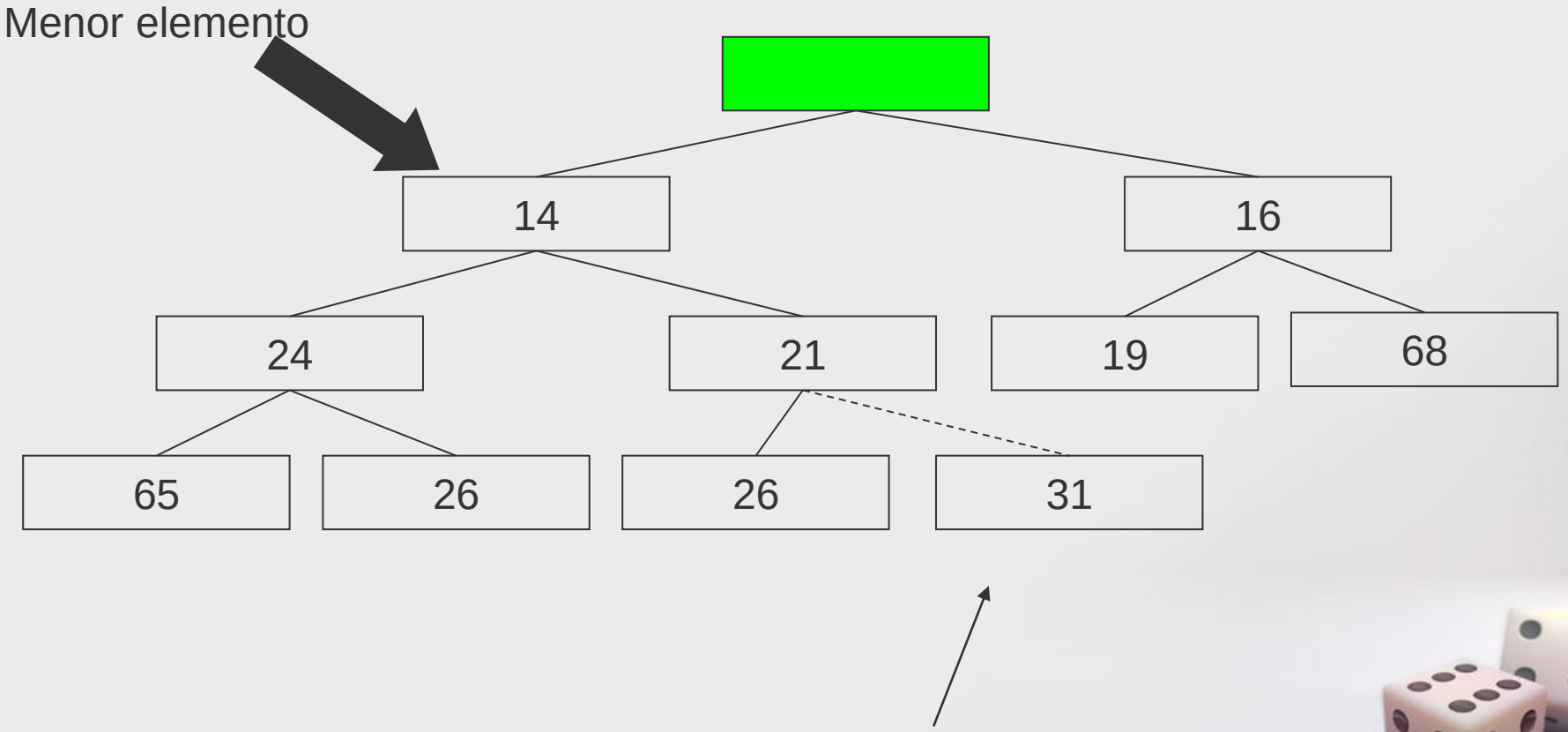
Heap Binária

- Operações: removeMinimo()



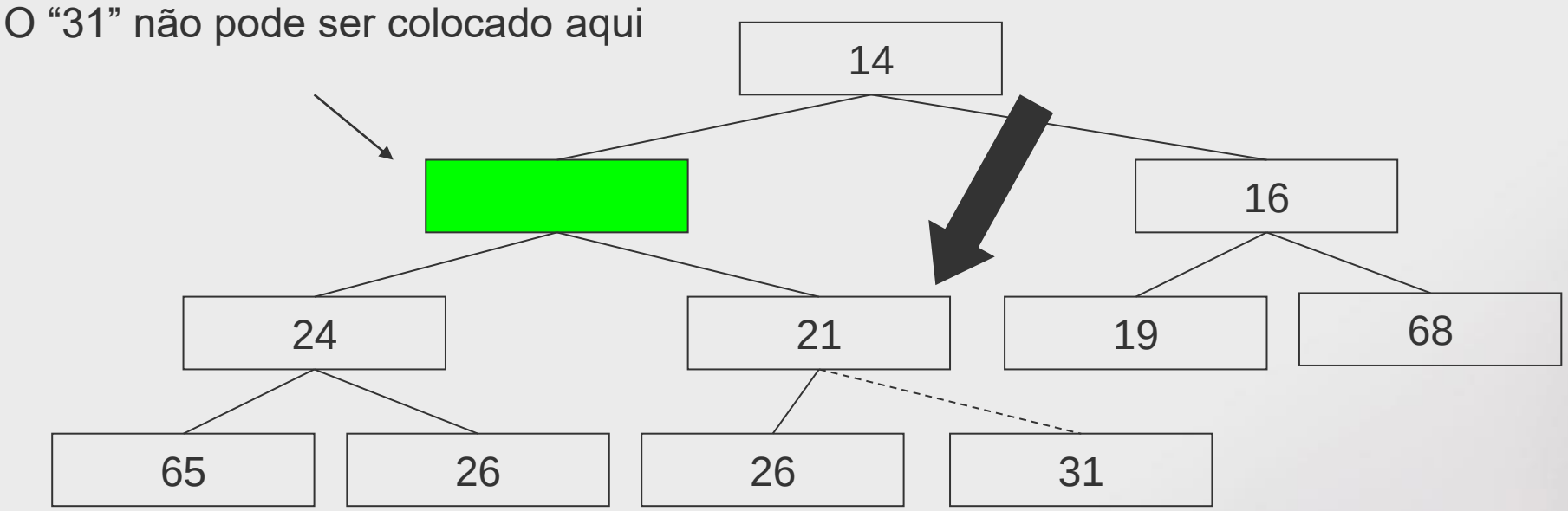
Heap Binária

- Operações: removeMinimo()



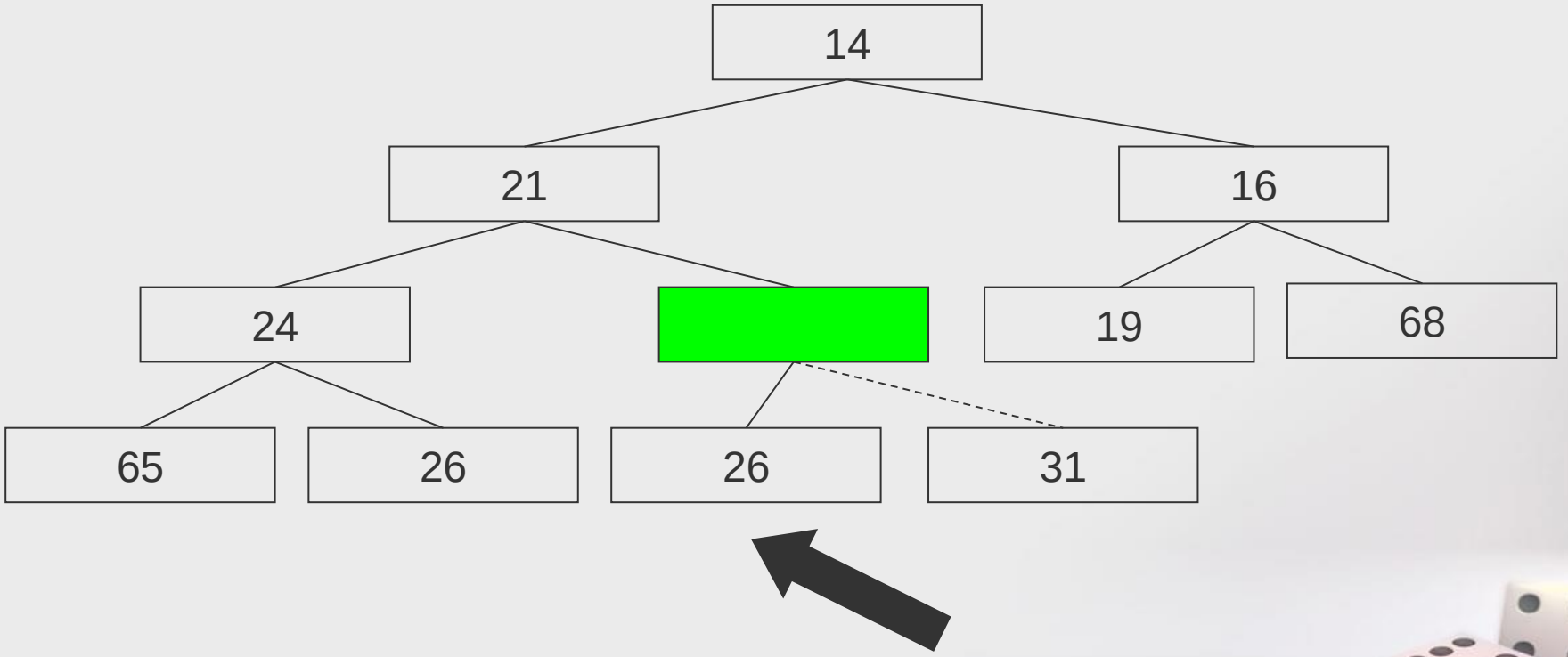
Heap Binária

- Operações: removeMinimo()



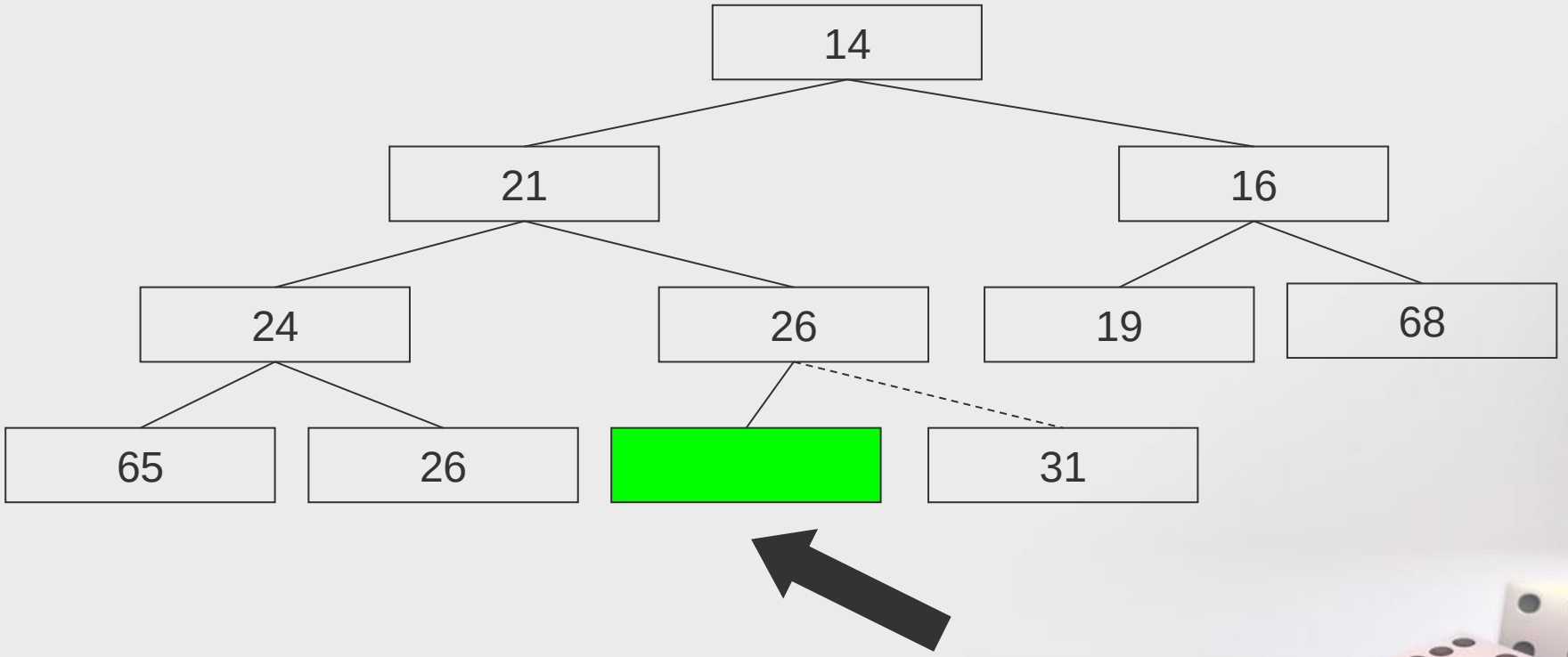
Heap Binária

- Operações: removeMinimo()



Heap Binária

- Operações: removeMinimo()

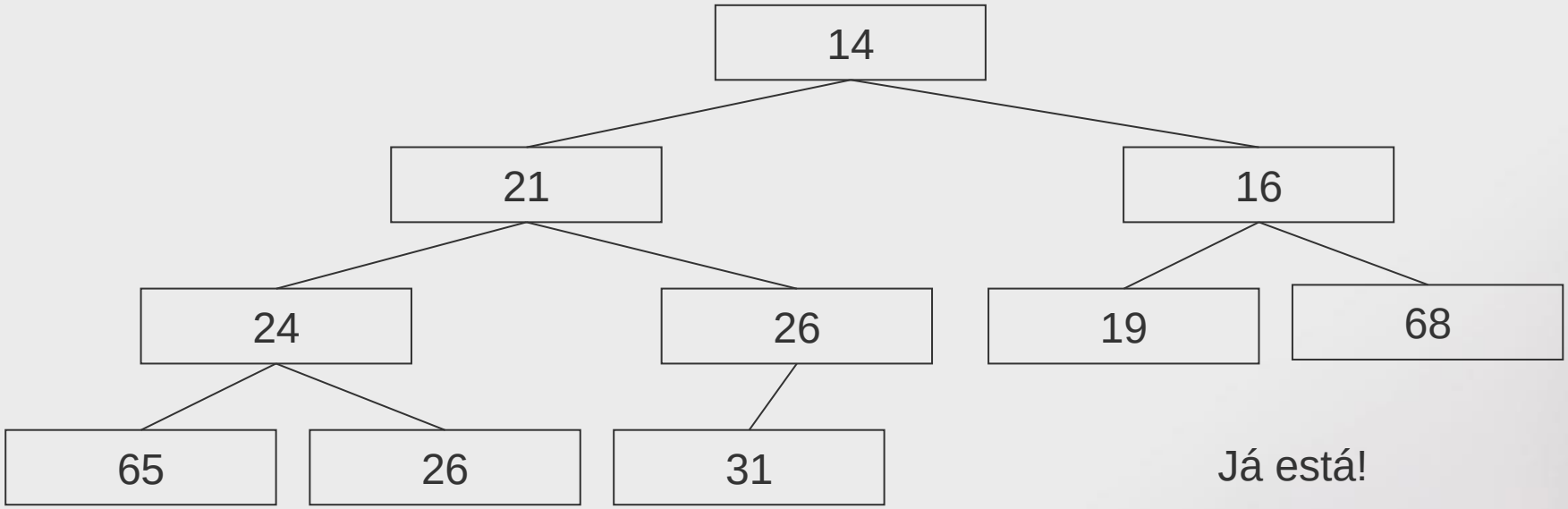


O nodo “31” pode, por fim, ser aqui colocado!



Heap Binária

- Operações: removeMinimo()



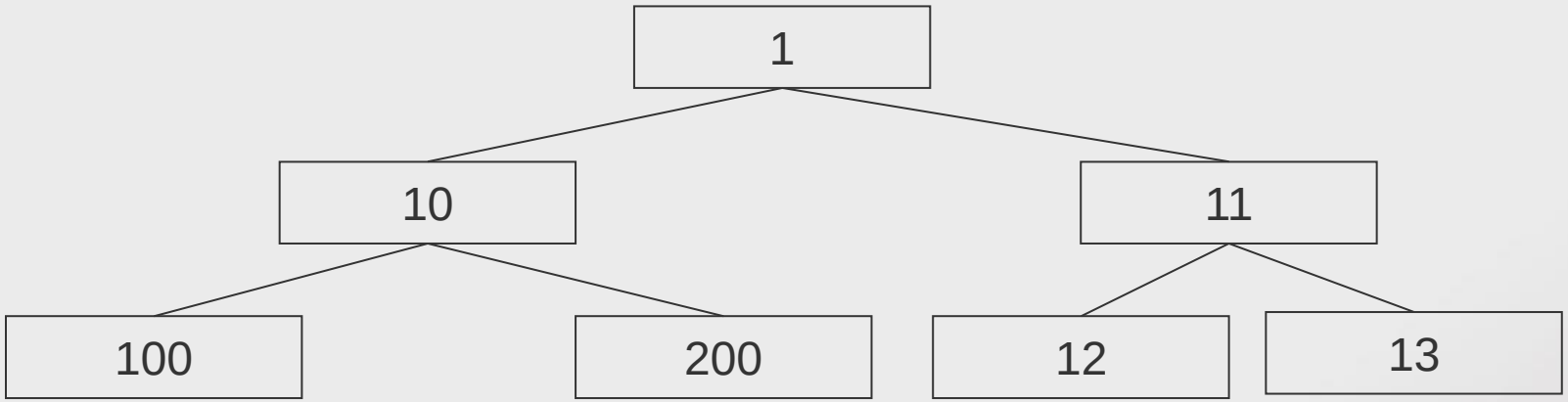
Já está!

Complexidade O(log N) máxima e média!



Heap Binária

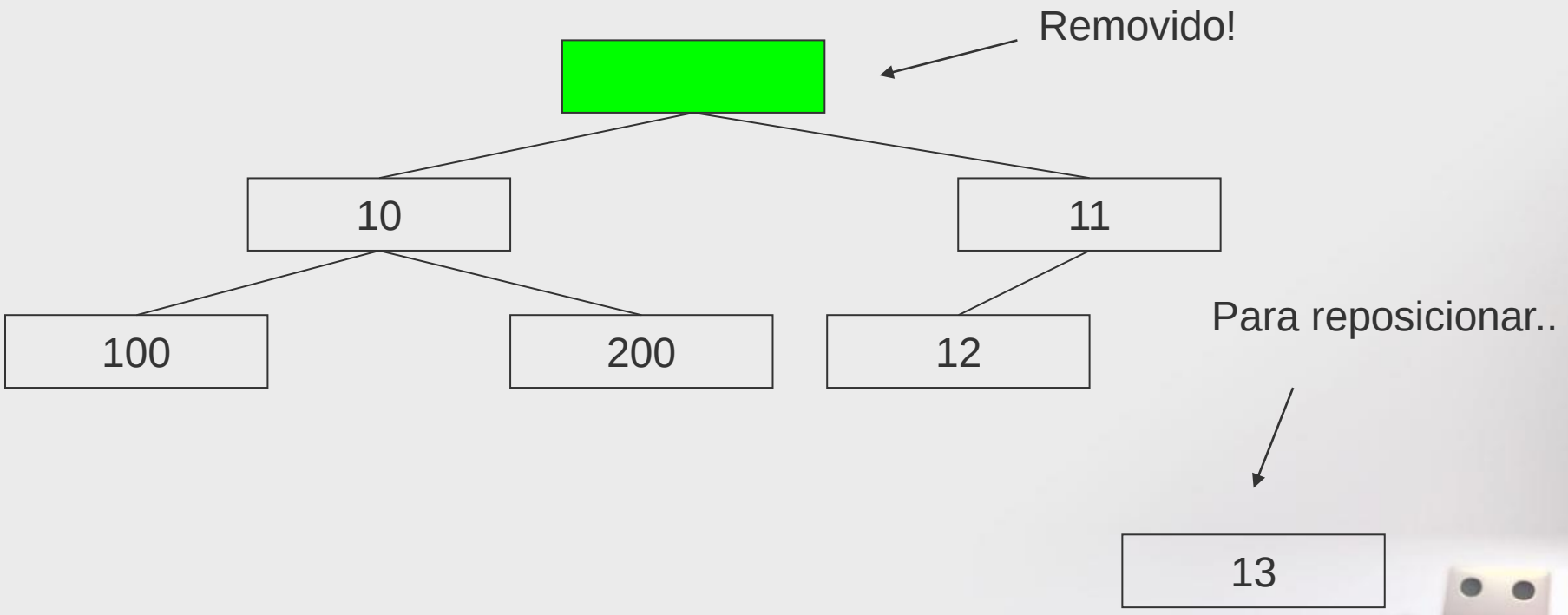
- removeMinimo() – Outro exemplo!





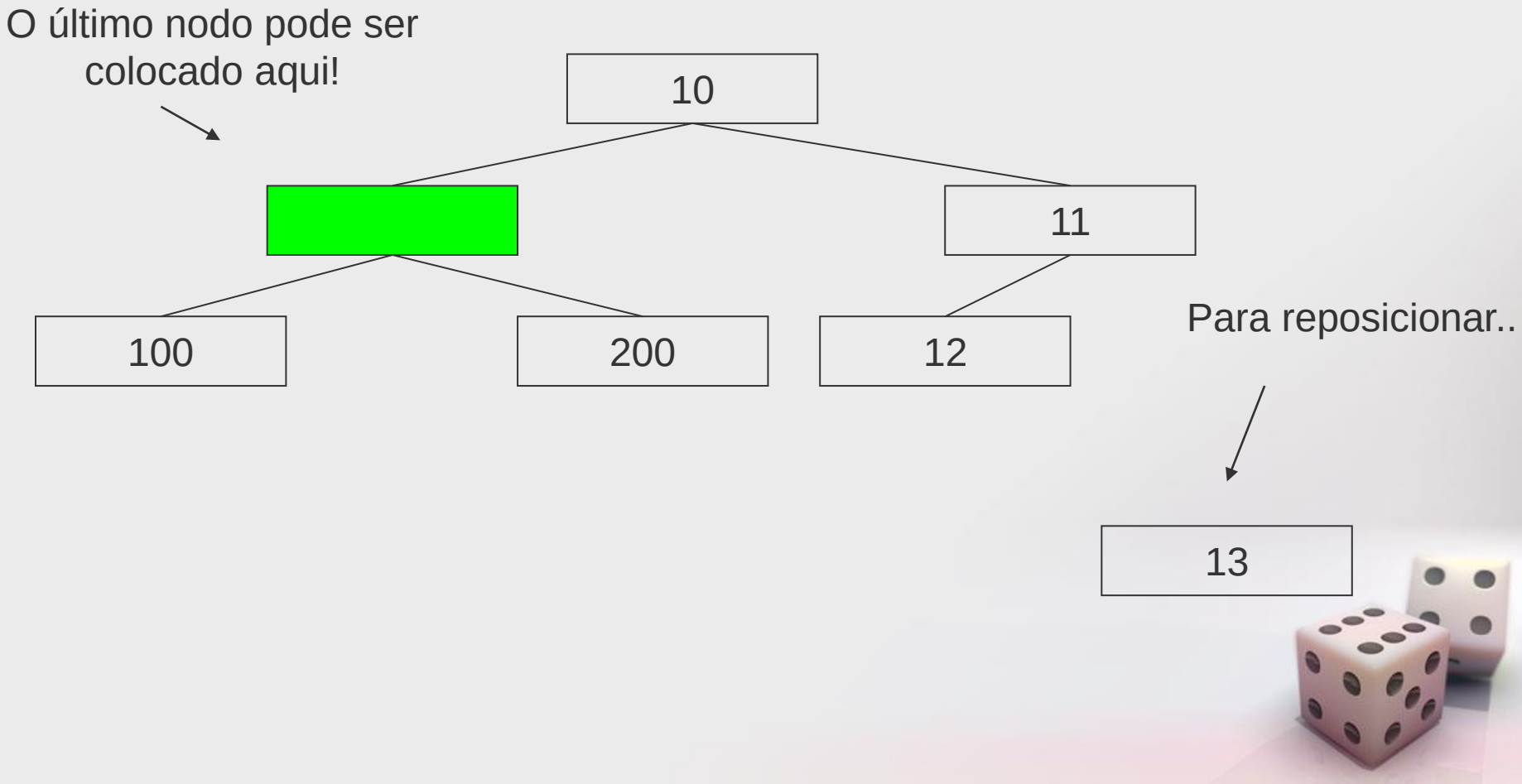
Heap Binária

- removeMinimo() – Outro exemplo!



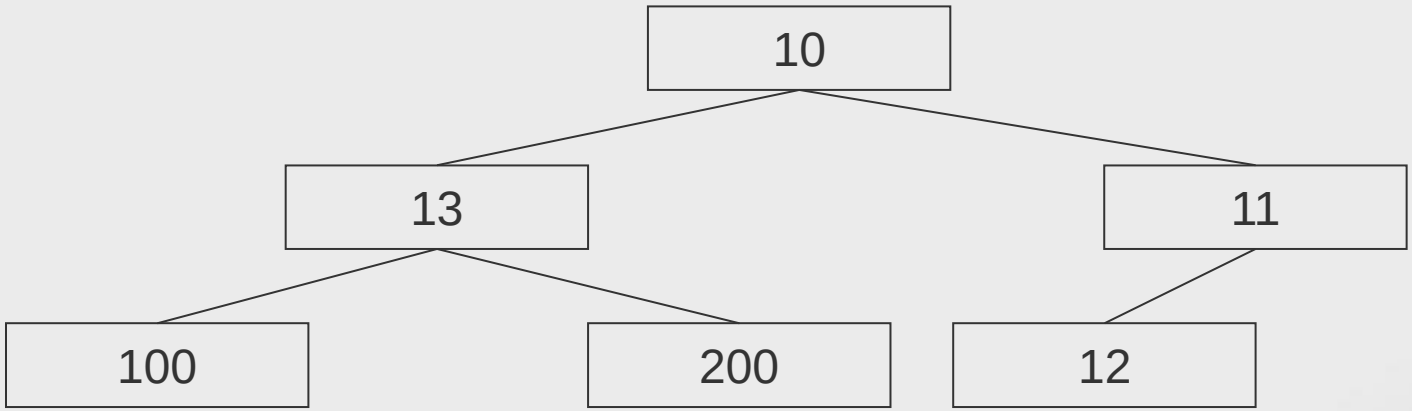
Heap Binária

- removeMinimo() – Outro exemplo!



Heap Binária

- removeMinimo() – Outro exemplo!



Final!

Heap Binária

- Inicialização de Heap Binária.
  - É possível inicializar de forma eficiente uma *heap* binária com N elementos.
  - Dada uma ABC qualquer, a ordem de *heap* pode ser obtida em O(N)
    - Inserir cada elemento individualmente numa heap vazia demora, naturalmente, O(N log N) no pior caso.



Heap Binária

- Inicialização- Ideia Básica
  - Se ambos os descendentes da raiz forem *heaps* binárias…
    - Então basta percolar para baixo a raiz e obtém-se uma heap binária.
  - Isto sugere o seguinte algoritmo recursivo:

```
inicializaHeap(raiz){
    inicializaHeap(raiz.esquerda);
    inicializaHeap(raiz.direita);
    percolaBaixo(raiz);
}
```



Heap Binária

- Em termos práticos, a recursão não é necessária:
  - Basta garantir que todos os nodos descendentes já foram processados…
  - Isso é obtido, chamando *percolarBaixo()* em cada nodo por ordem inversa de nível!
    - *Não é necessário percolar as folhas*



Heap Binária

Inicialização:

```
private void buildHeap( )
    {
    for( int i = currentSize / 2; i > 0; i-- )
    percolateDown( i );
    }
```



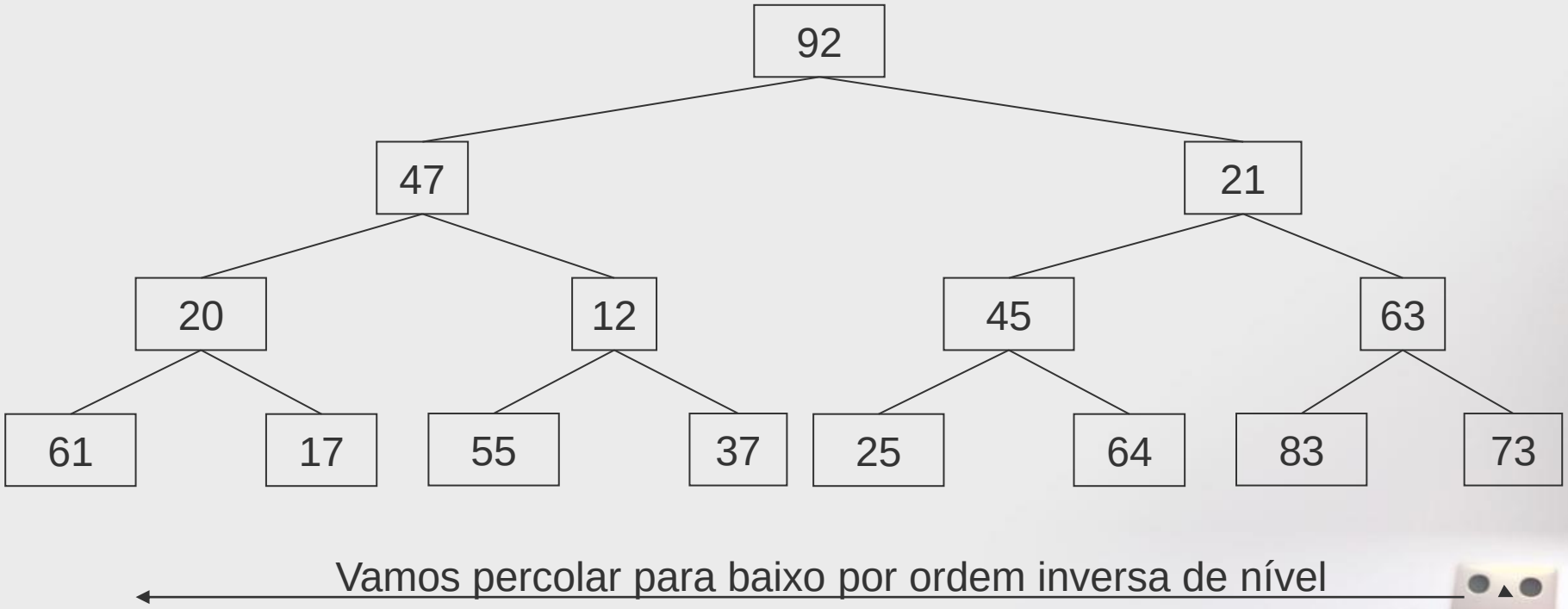
Heap Binária

- Exemplo: Inicialização



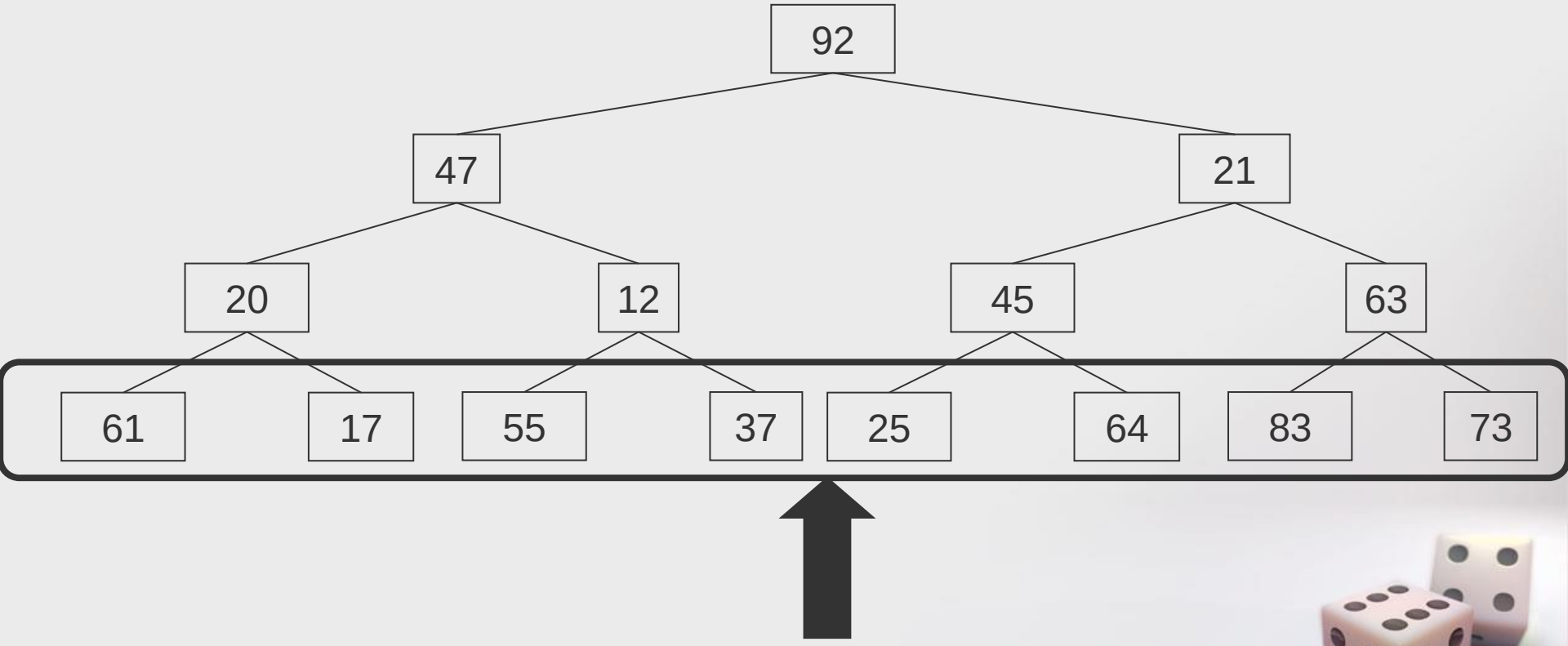
Heap Binária

- Exemplo: inicialização



Heap Binária

- Exemplo: inicialização

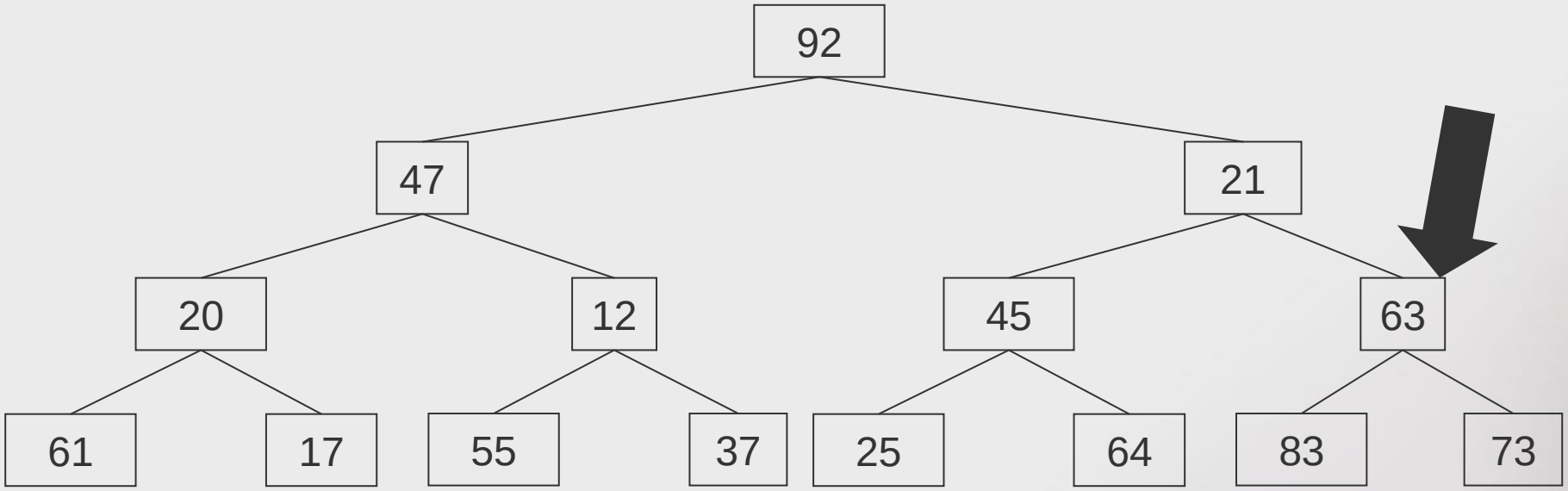


Não há qualquer efeito nas folhas



Heap Binária

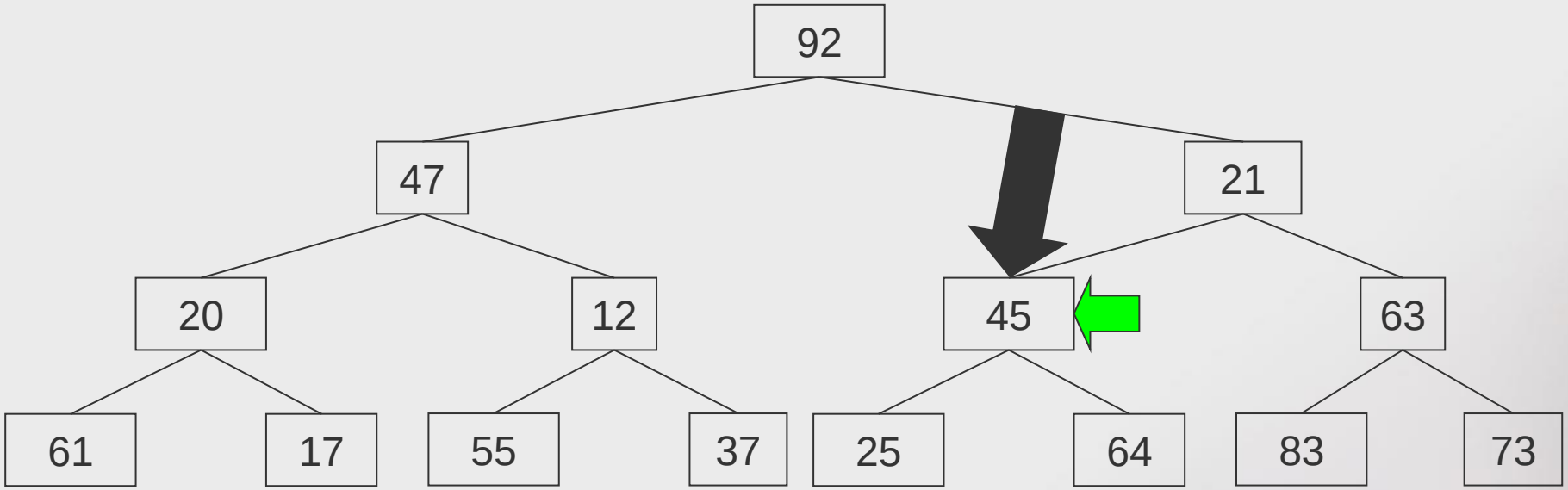
- Exemplo: inicialização





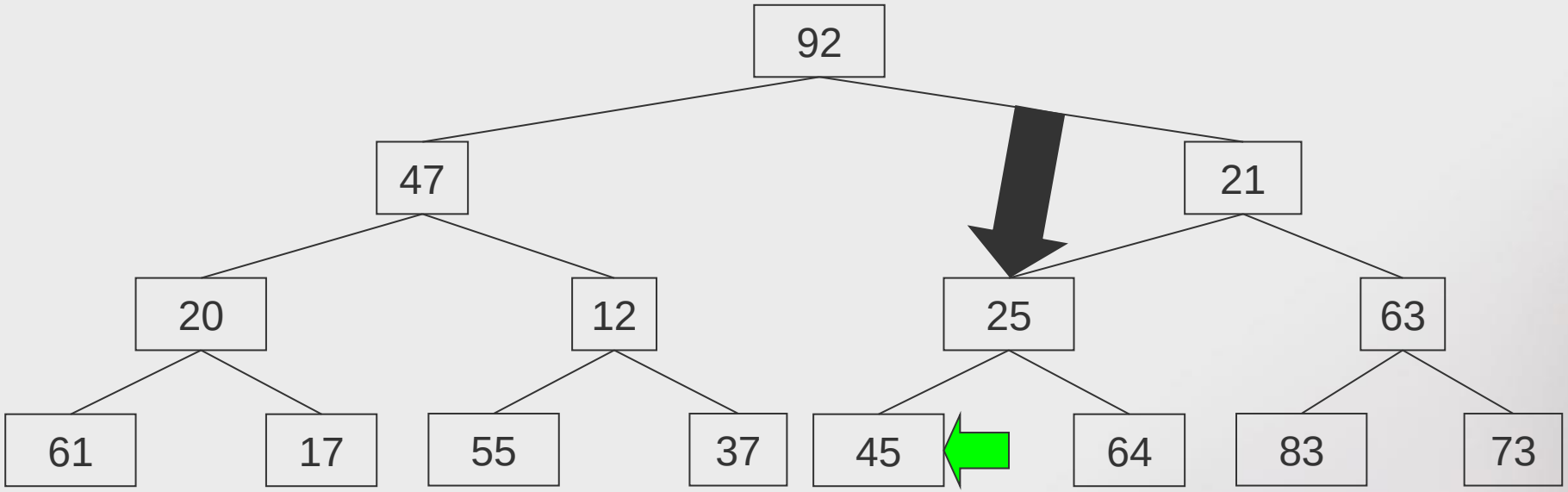
Heap Binária

- Exemplo: inicialização



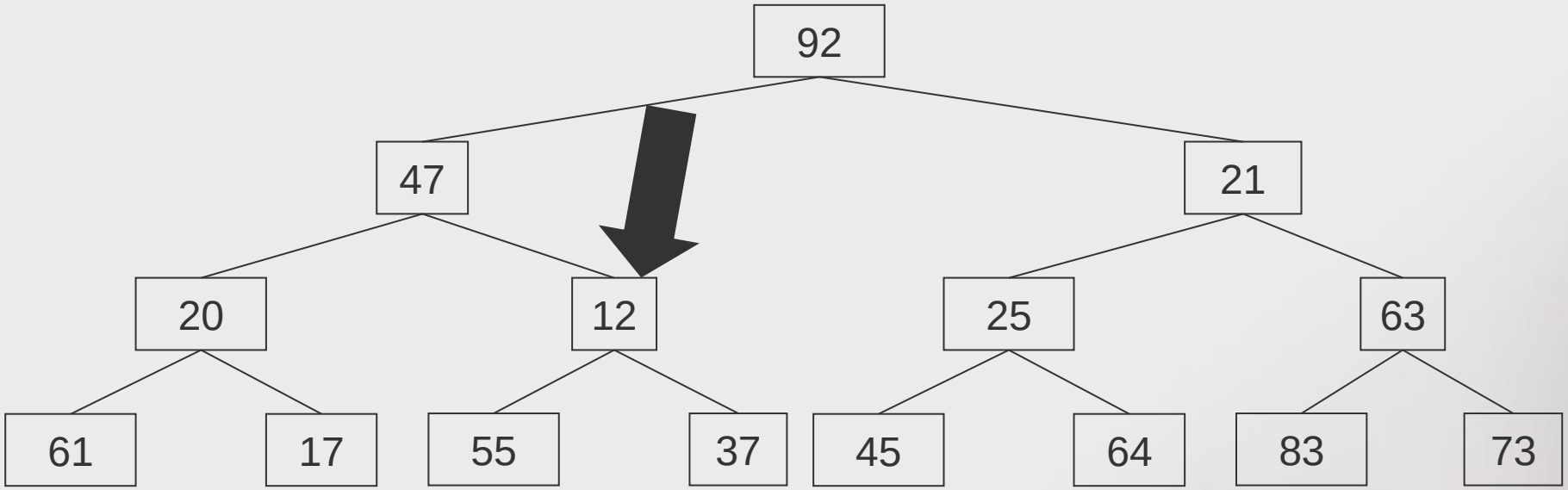
Heap Binária

- Exemplo: inicialização



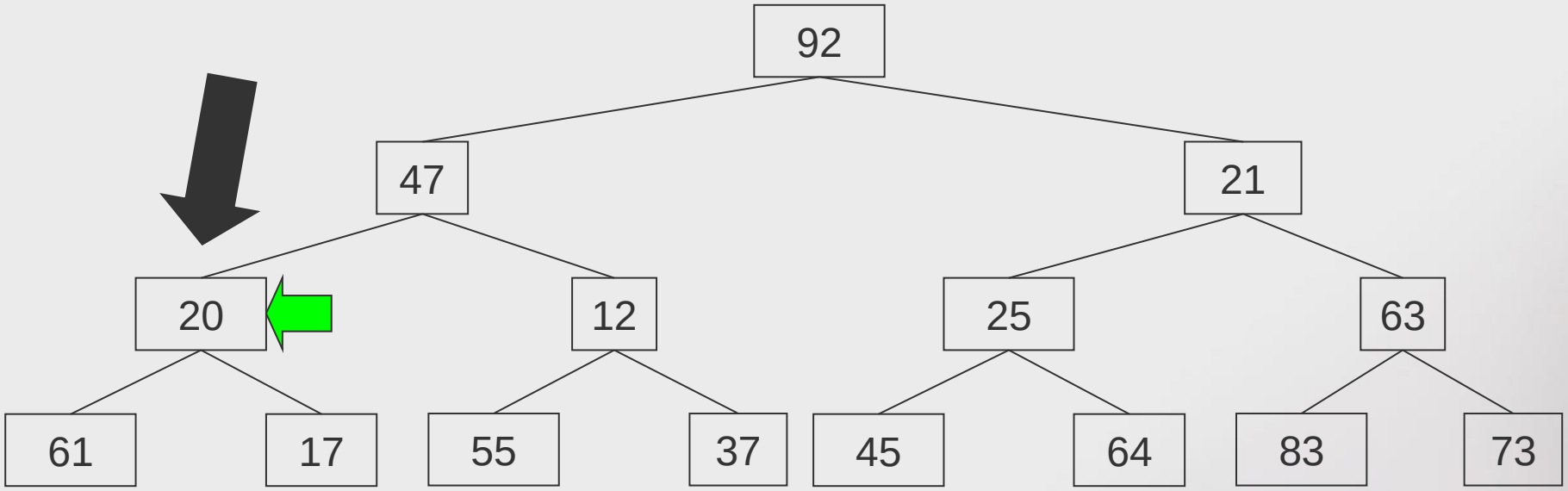
Heap Binária

- Exemplo: inicialização



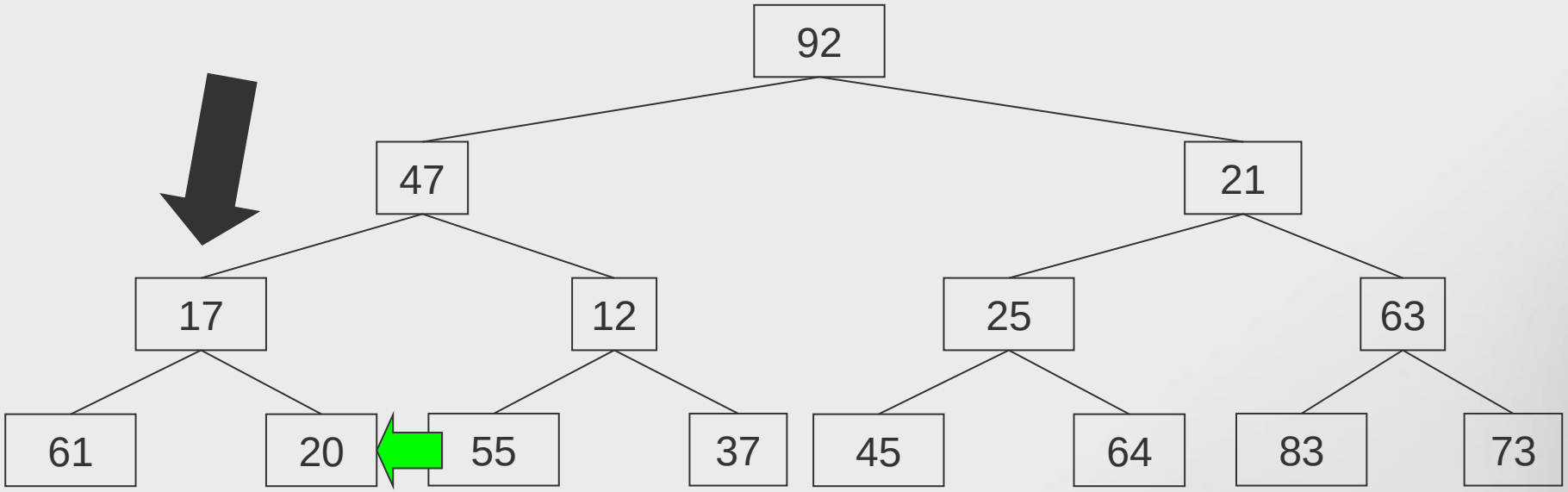
Heap Binária

- Exemplo: inicialização



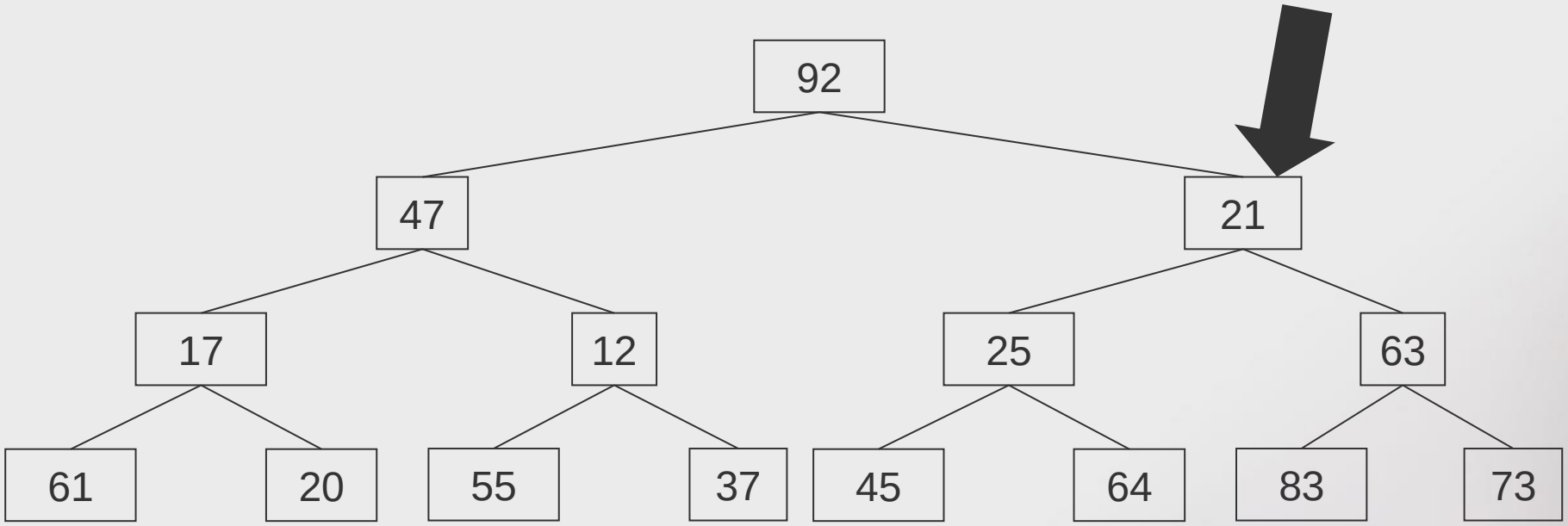
Heap Binária

- Exemplo: inicialização



Heap Binária

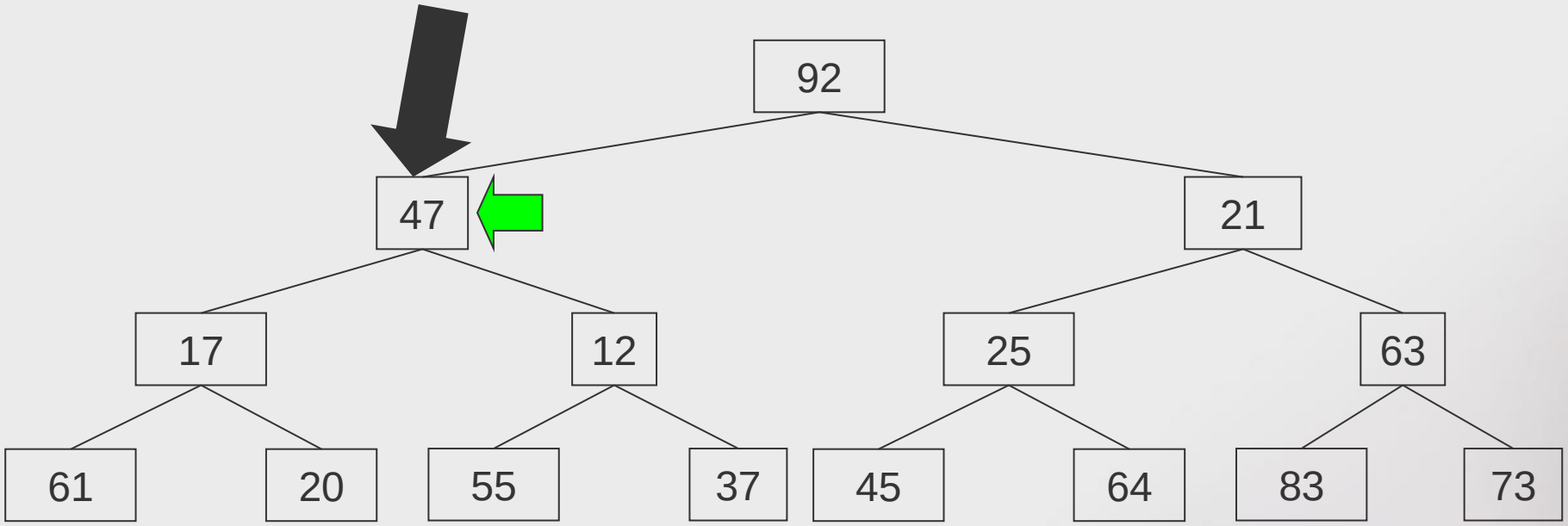
- Exemplo: inicialização





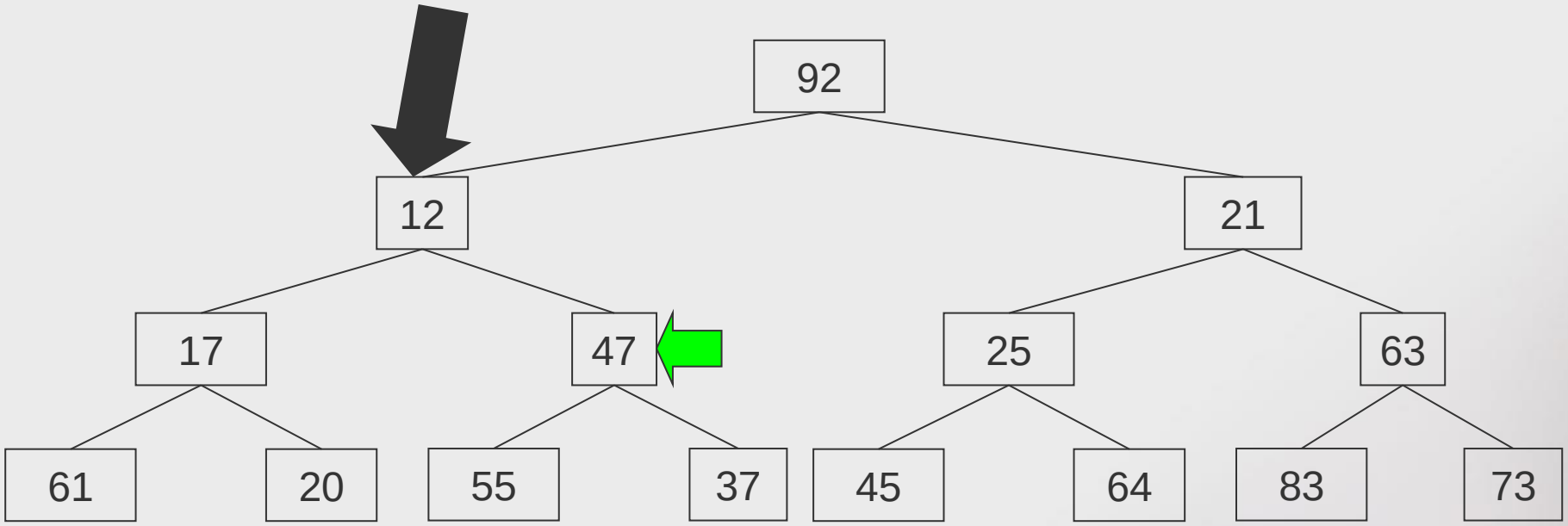
Heap Binária

- Exemplo: inicialização



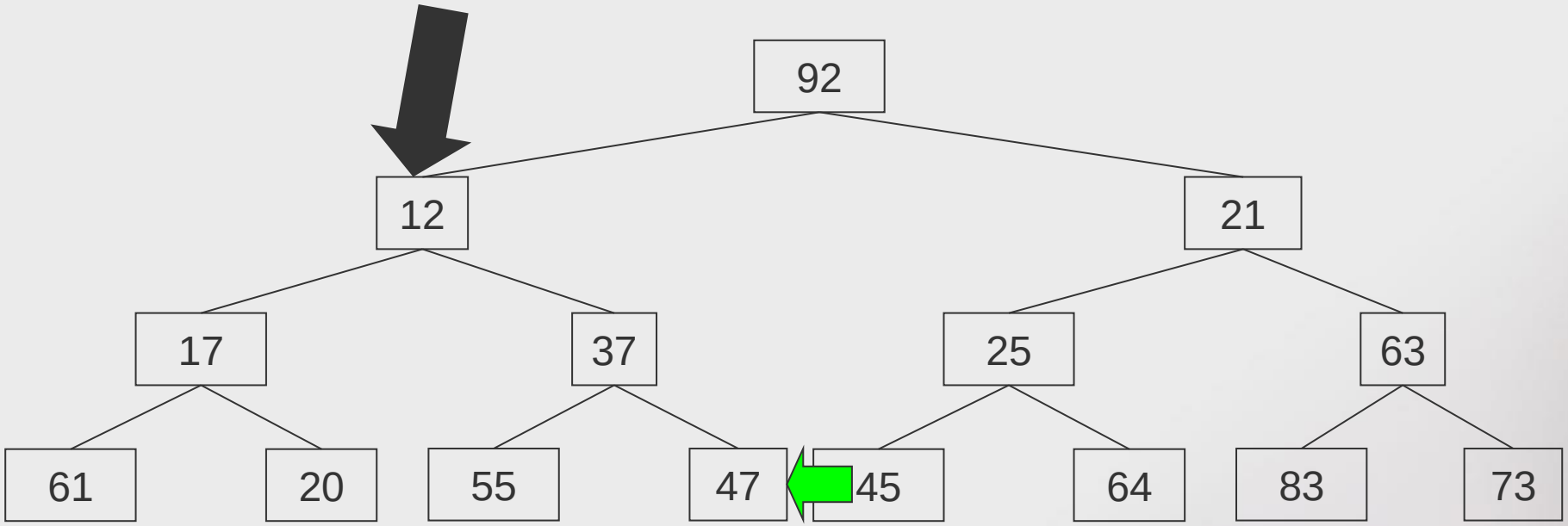
Heap Binária

- Exemplo: inicialização



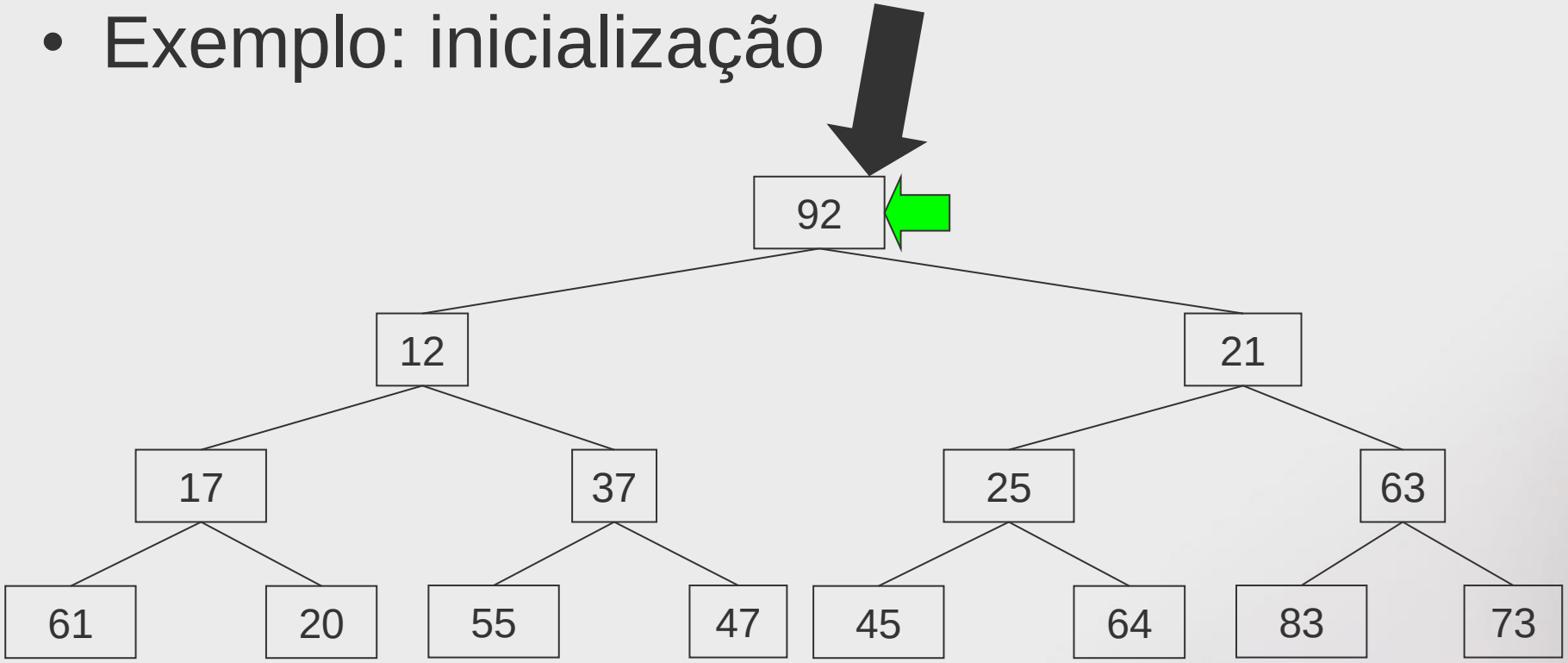
Heap Binária

- Exemplo: inicialização



Heap Binária

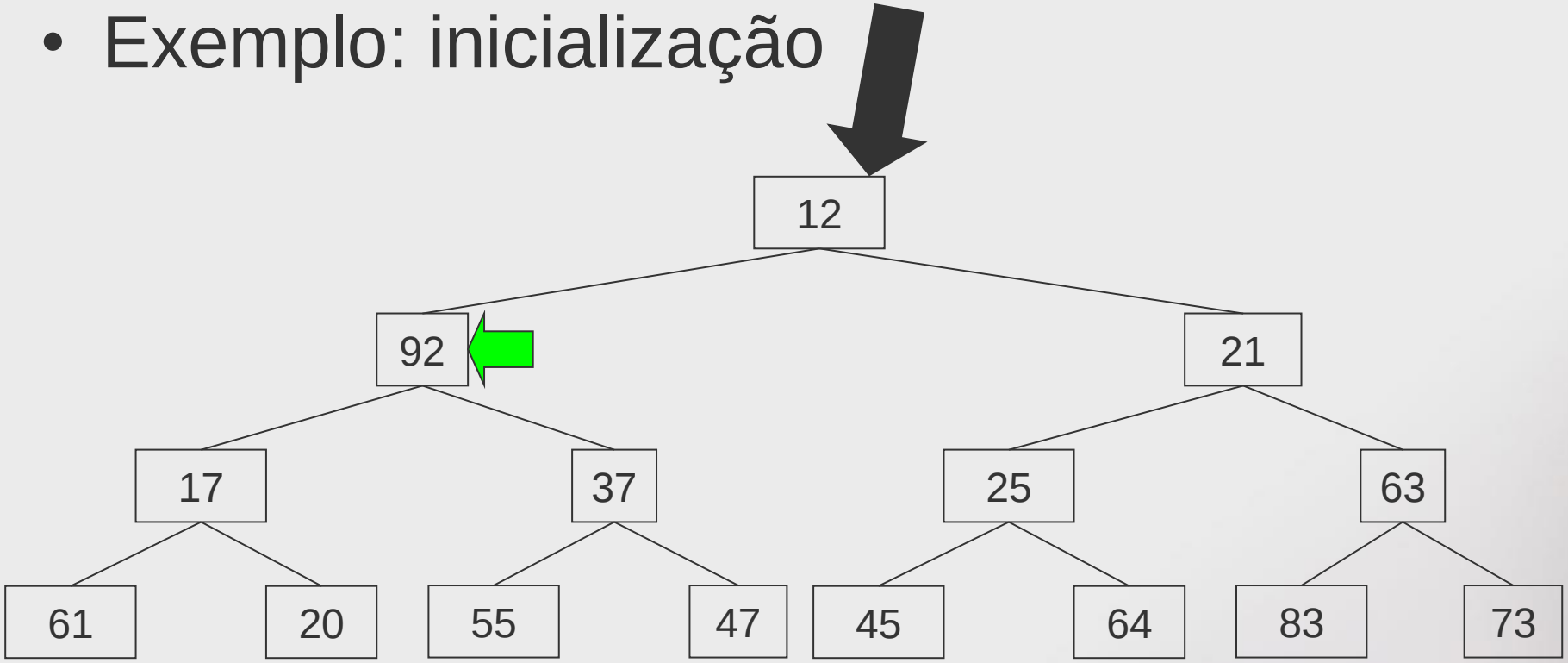
- Exemplo: inicialização





Heap Binária

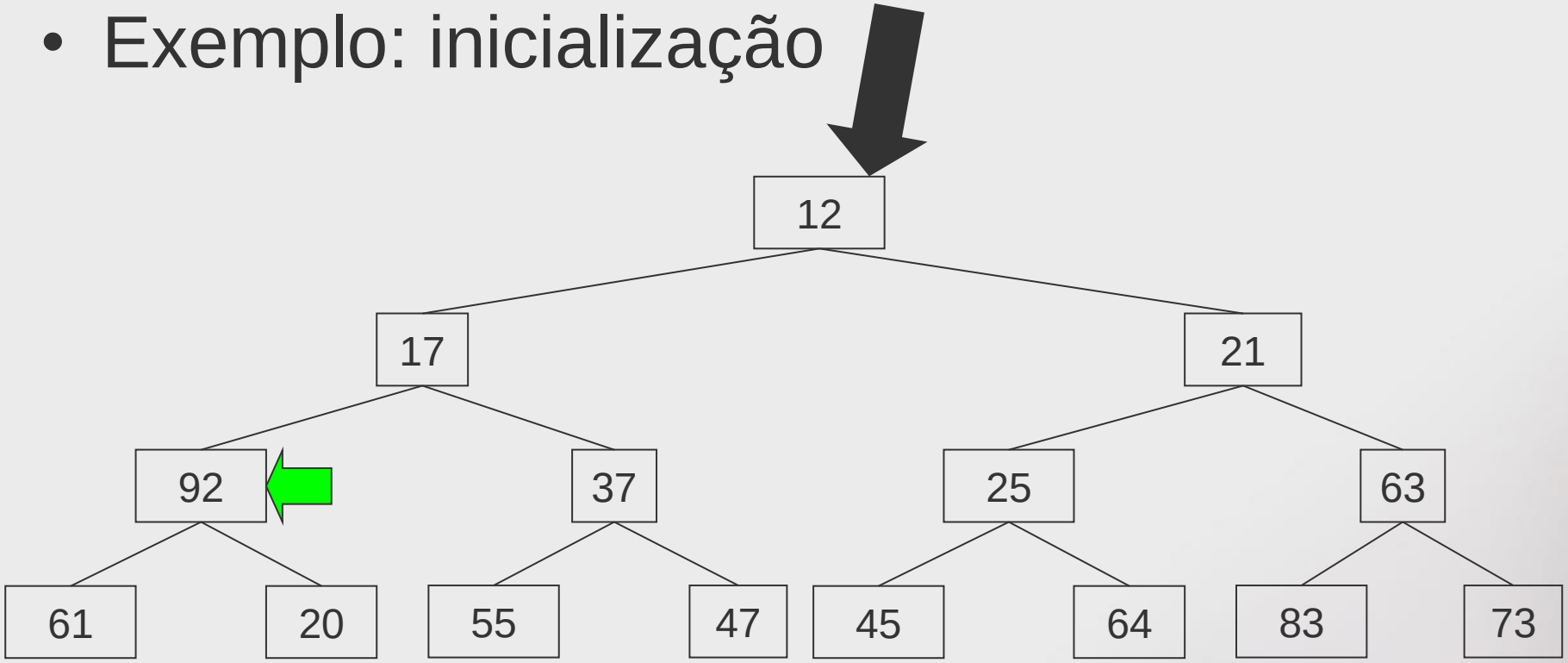
- Exemplo: inicialização





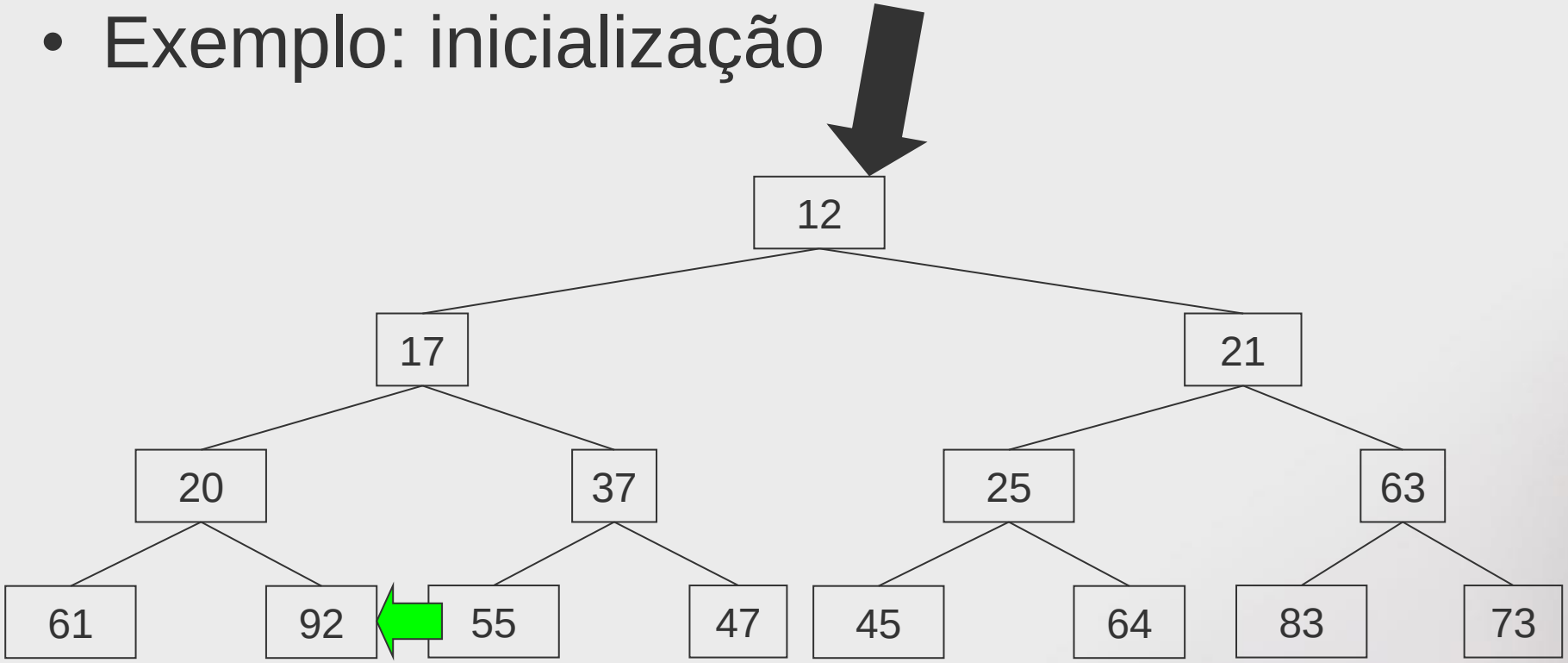
Heap Binária

- Exemplo: inicialização



Heap Binária

- Exemplo: inicialização



Heap Binária

- Aplicações:
  - Podem ser elencadas várias aplicações onde as Heaps são uma parte fundamental de um algoritmo eficiente.
    - *Construção de árvores de Huffman*
    - *Algoritmos de grafos (p.ex. Dijkstra e Prim)*
    - *Cálculo da mediana*
    - *Heap Sort*



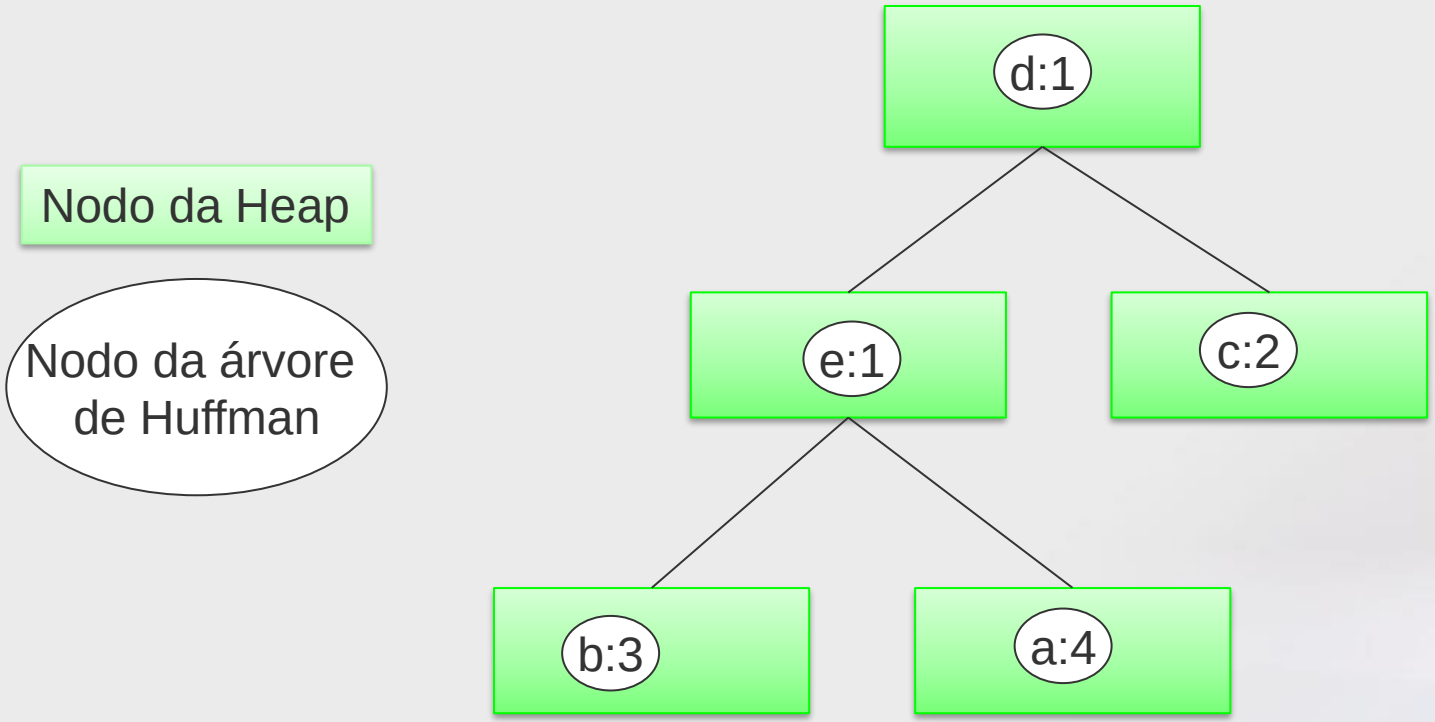
Heap Binária

- Exemplo: Árvore de Huffman
  - Em cada passo do algoritmo escolhemos os dois subconjuntos (sub-árvores) de caracteres com menos elementos, que são combinados para formar uma nova sub-árvore.
  - As sub-árvores podem ser guardadas numa heap durante este processo, fazendo com o a identificação dos dois elementos a combinar seja feita simplesmente removendo os dois menores valores da heap.



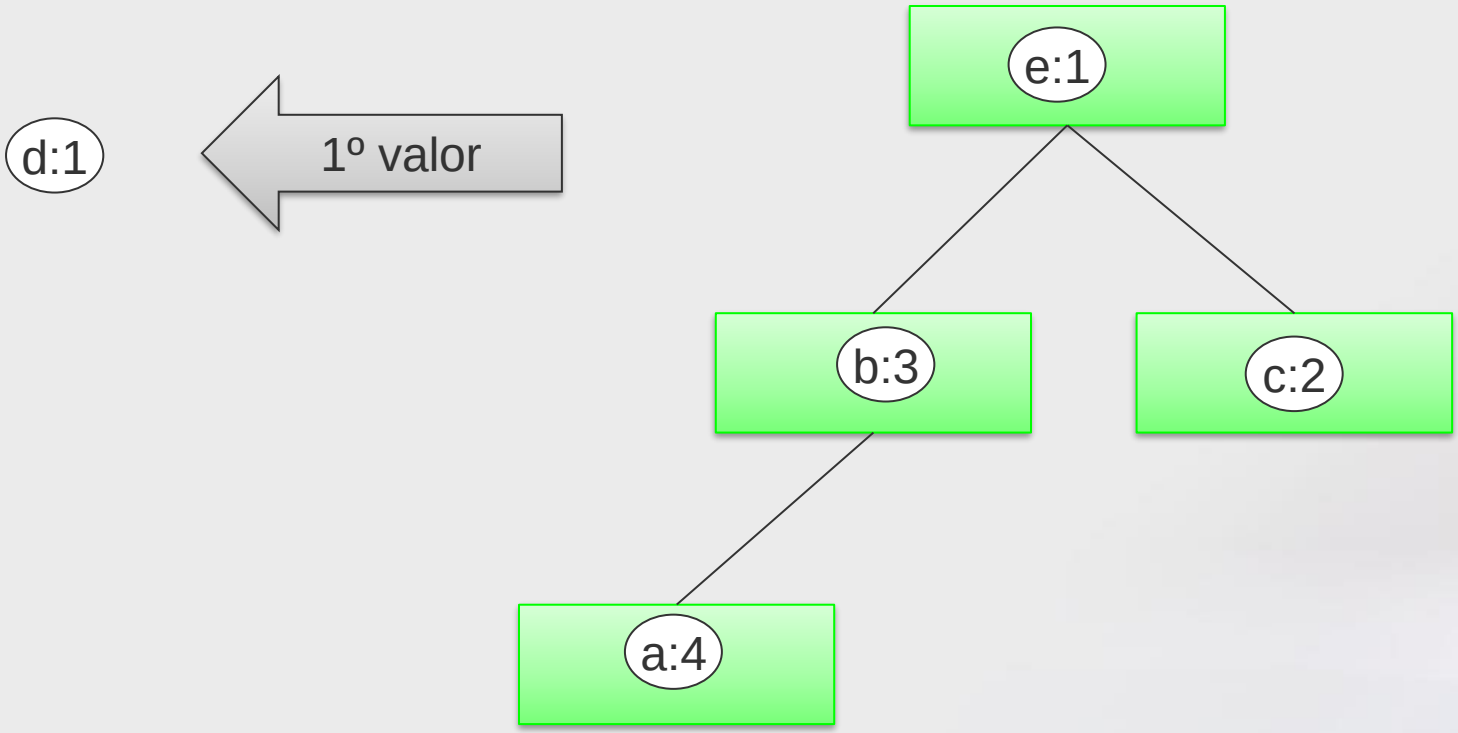
Heap Binária

- **Exemplo:** Árvore de Huffman criada com *Heap*
- **Palavra:** `aaaabbbccde`



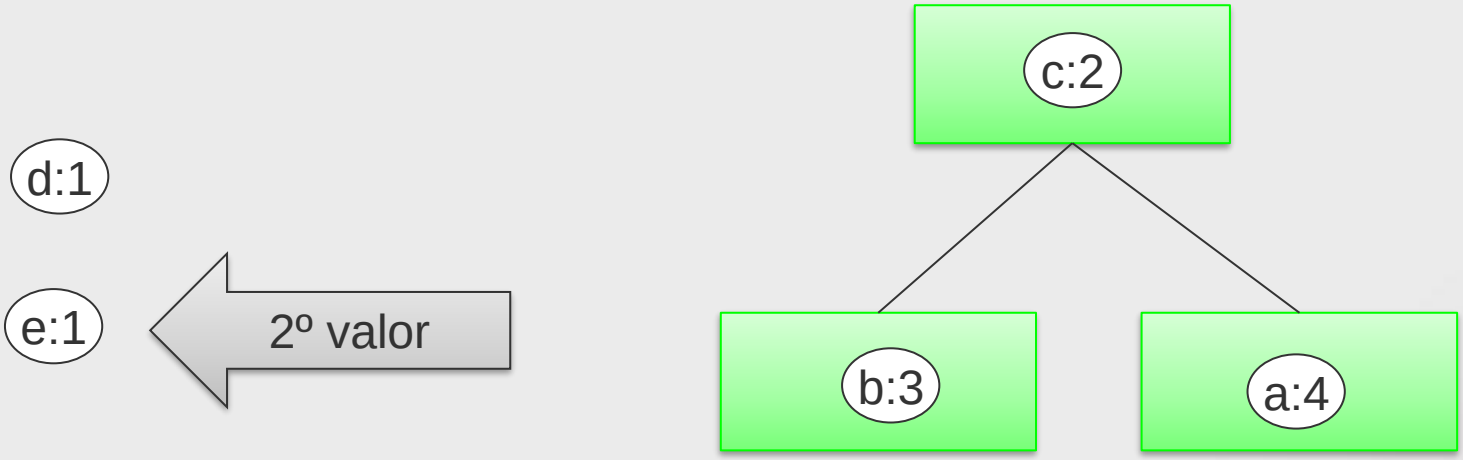
Heap Binária

- **Exemplo:** Árvore de Huffman criada com Heap
  - São removidos os dois menores valores…



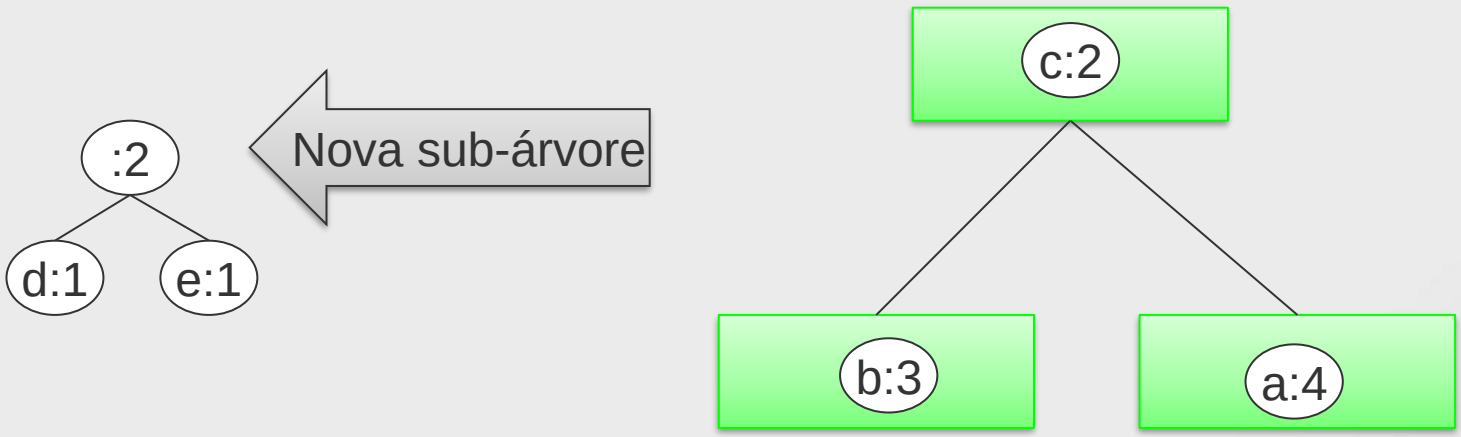
Heap Binária

- Exemplo: Árvore de Huffman criada com Heap
  - São removidos os dois menores valores…



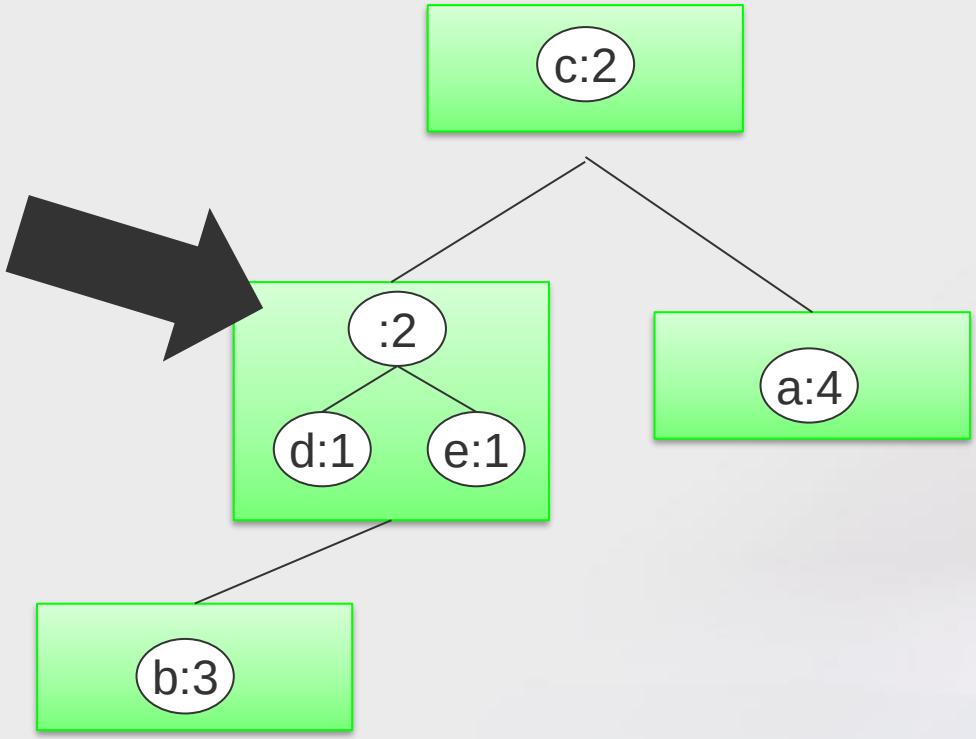
Heap Binária

- Exemplo: Árvore de Huffman com Heap
  - São combinados numa nova sub-árvore com 2 caracteres…



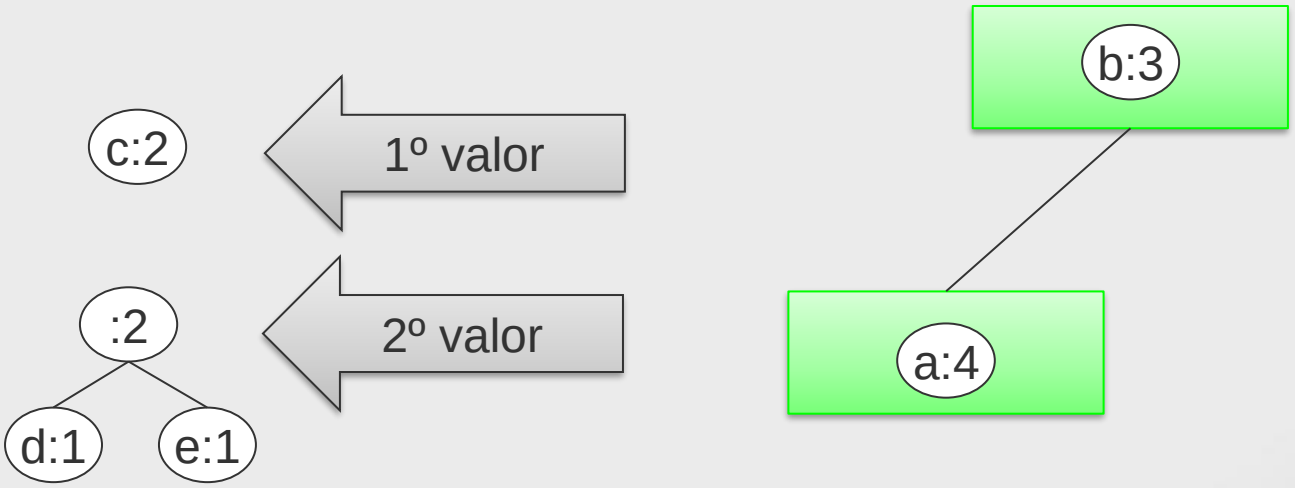
Heap Binária

- Exemplo: Árvore de Huffman com Heap
  - …Que é inserida na Heap!



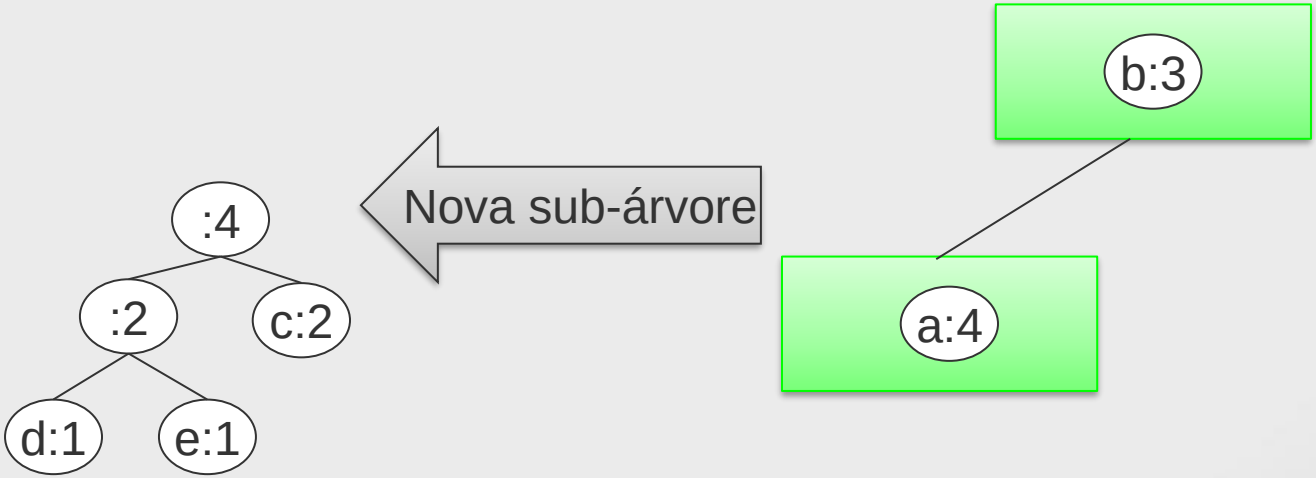
Heap Binária

- Exemplo: Árvore de Huffman com Heap
  - O processo repete-se até a árvore de Huffman estar completa!



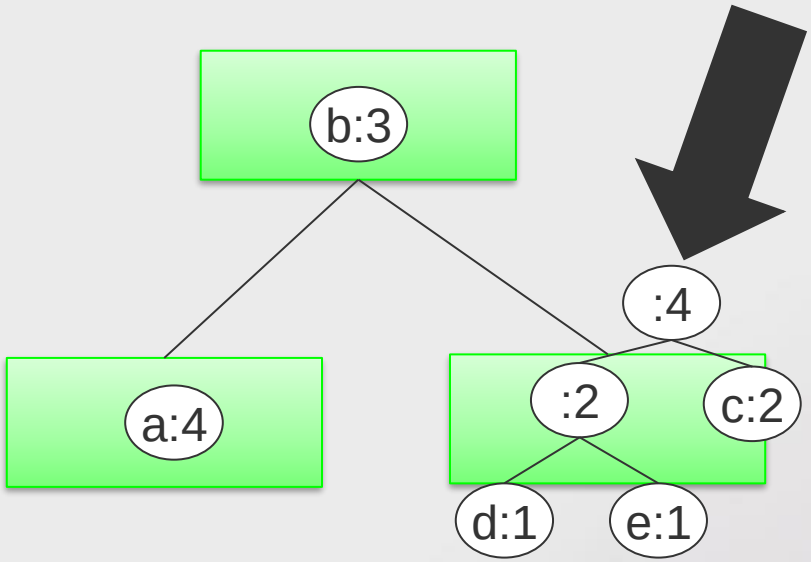
Heap Binária

- Exemplo: Árvore de Huffman com Heap
  - O processo repete-se até a árvore de Huffman estar completa!



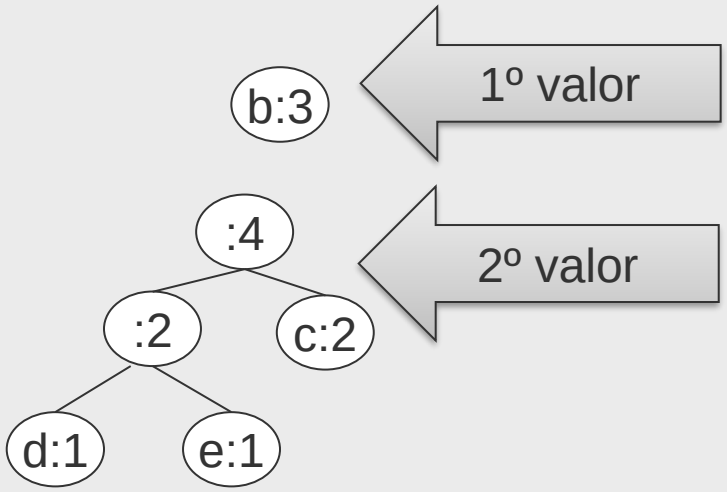
Heap Binária

- Exemplo: Árvore de Huffman com Heap
  - O processo repete-se até a árvore de Huffman estar completa!



Heap Binária

- Exemplo: Árvore de Huffman com Heap
  - O processo repete-se até a árvore de Huffman estar completa!

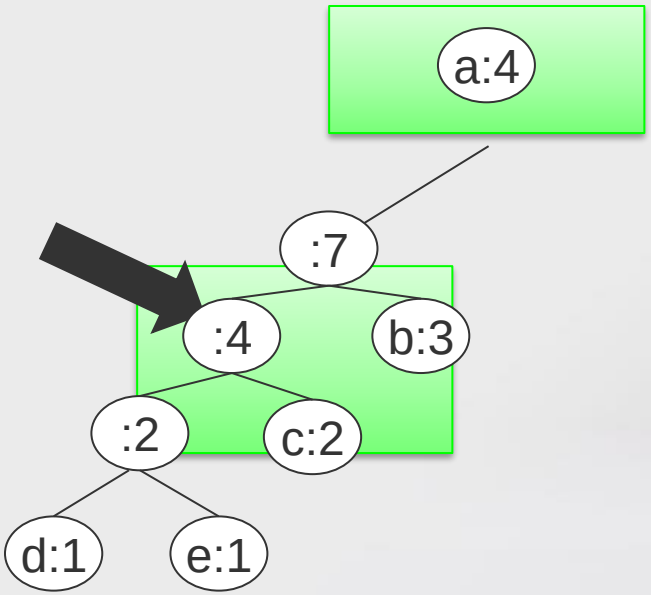






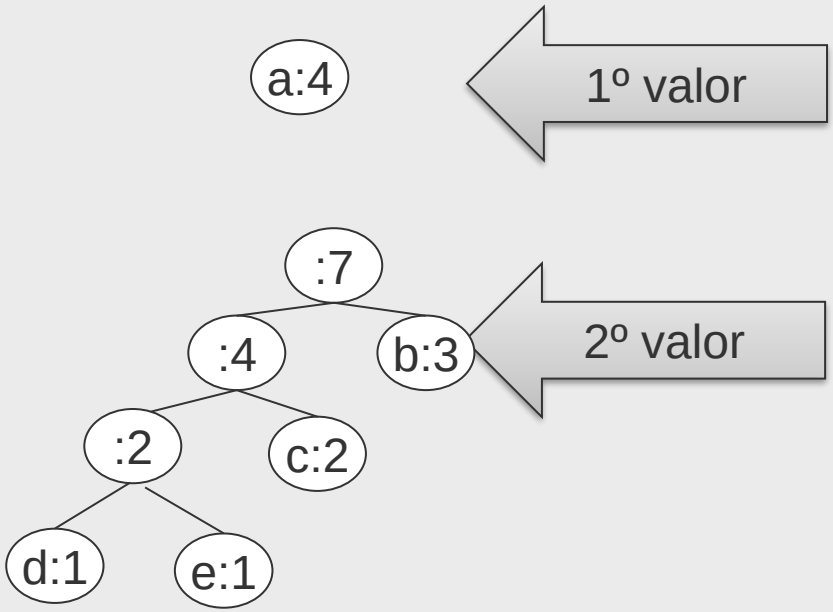
Heap Binária

- Exemplo: Árvore de Huffman com Heap
  - O processo repete-se até a árvore de Huffman estar completa!



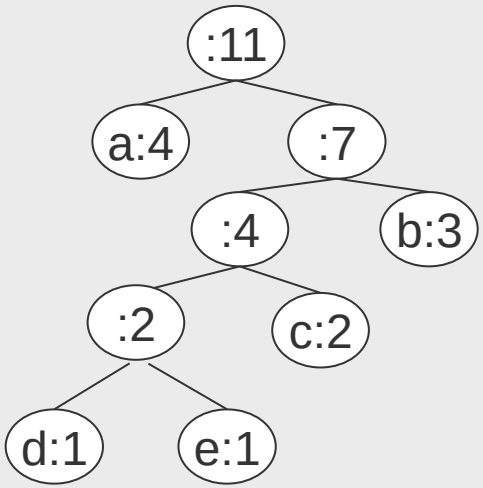
Heap Binária

- Exemplo: Árvore de Huffman com Heap
  - O processo repete-se até que reste apenas um nodo na heap.



Heap Binária

- Exemplo: Árvore de Huffman com Heap
  - A árvore de Huffman está completa.

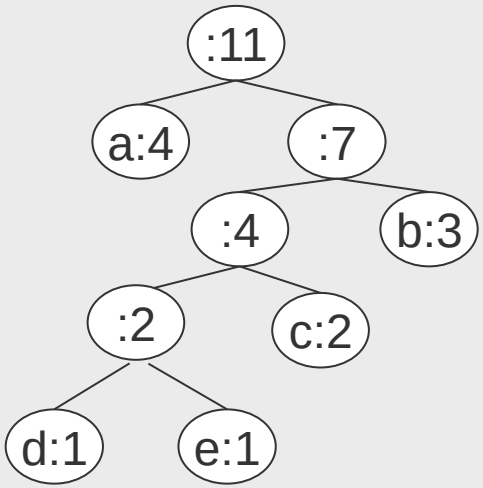


Caracter	Codificação
a	0
b	11
c	101
d	1000
e	1001



Heap Binária

- Exemplo: Árvore de Huffman com Heap
  - A árvore de Huffman está completa.



Caracter	Codificação
a	0
b	11
c	101
d	1000
e	1001

`aaaabbbccde =`
`00001111111011011000 1001`

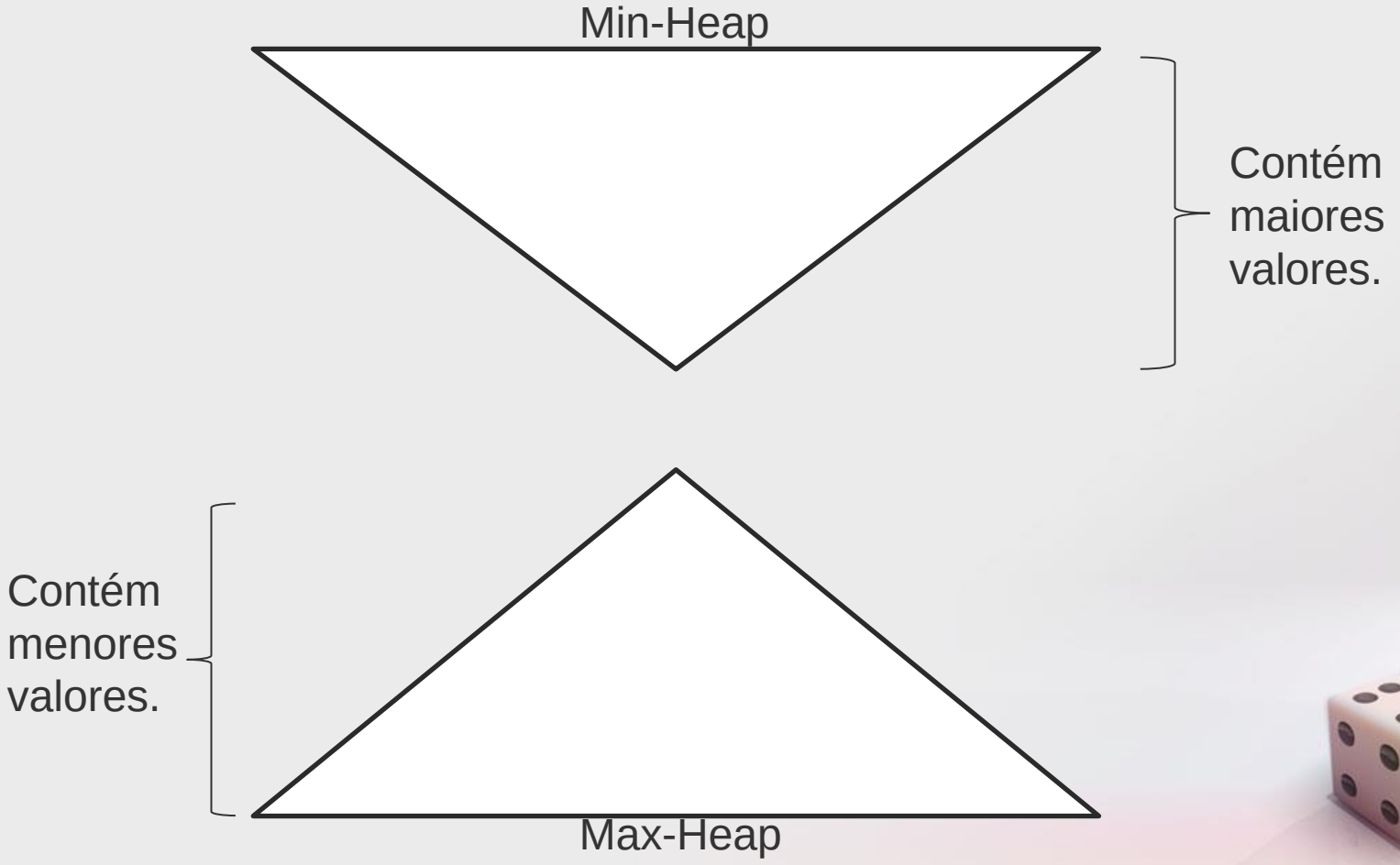
Heap Binária

- Exemplo: Cálculo da mediana de uma sequência de valores
  - São usadas duas heaps, tendo cada uma delas sempre metade (+1 ou -1, no caso de um número impar) dos valores da sequência.
  - Uma das heaps (uma MAX-HEAP) tem a metade dos valores mais pequenos. A outra heap (uma MIN-HEAP) tem a metade dos valores maiores.
  - Sempre que é inserido um valor, podem ser transferidos valores de uma *heap* para outra para assegurar a manutenção da distribuição equitativa.
  - A mediana é obtida analisando os valores no topo de ambas as heaps.



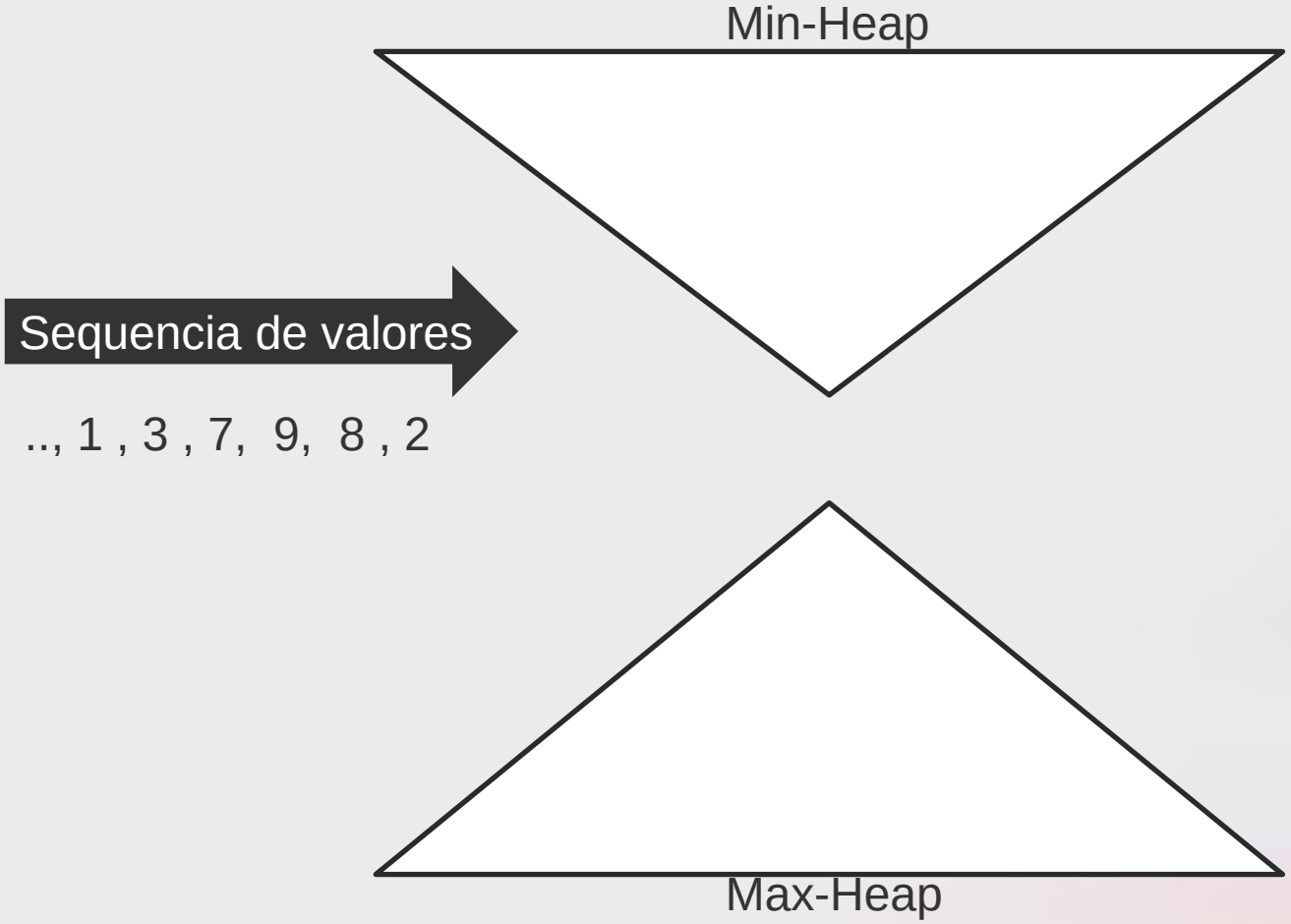
Heap Binária

- Exemplo: Cálculo da mediana de uma sequência de valores



Heap Binária

- Exemplo: Cálculo da mediana de uma sequência de valores

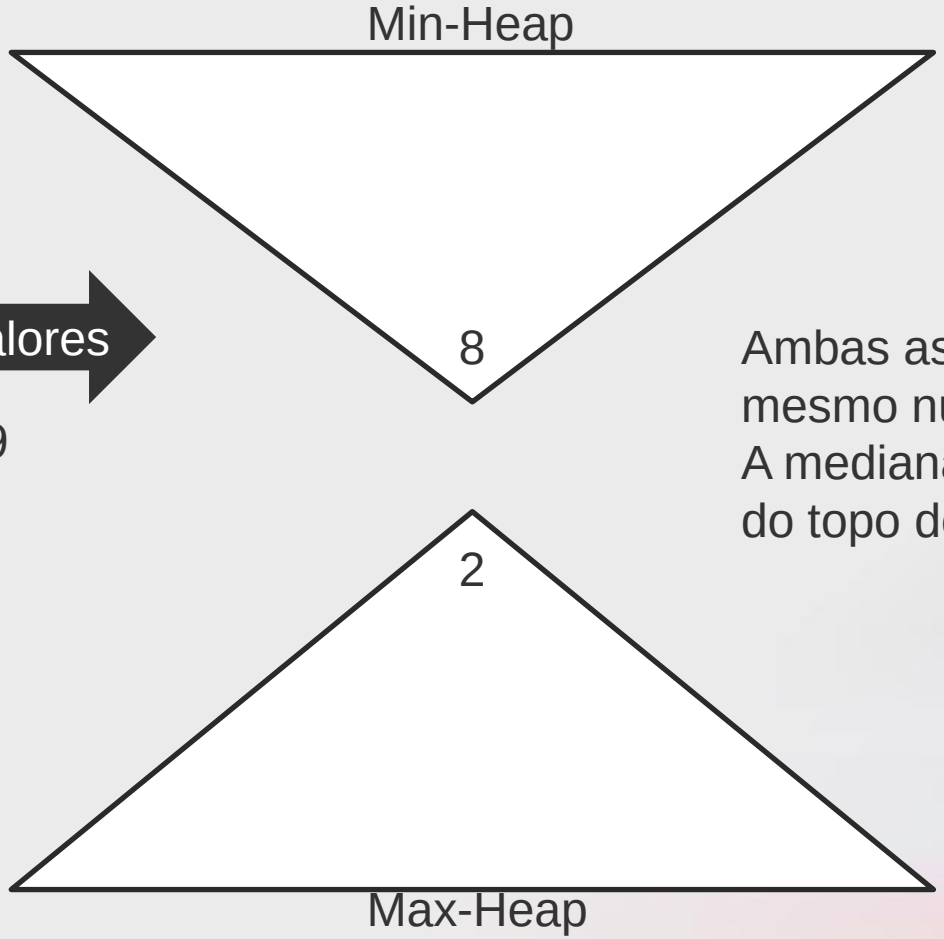


Heap Binária

- Exemplo: Cálculo da mediana de uma sequência de valores

Sequencia de valores

…, 4, 1 , 3 , 7, 9



Ambas as heaps têm o mesmo número de valores. A mediana é o valor médio do topo de cada heap (5)

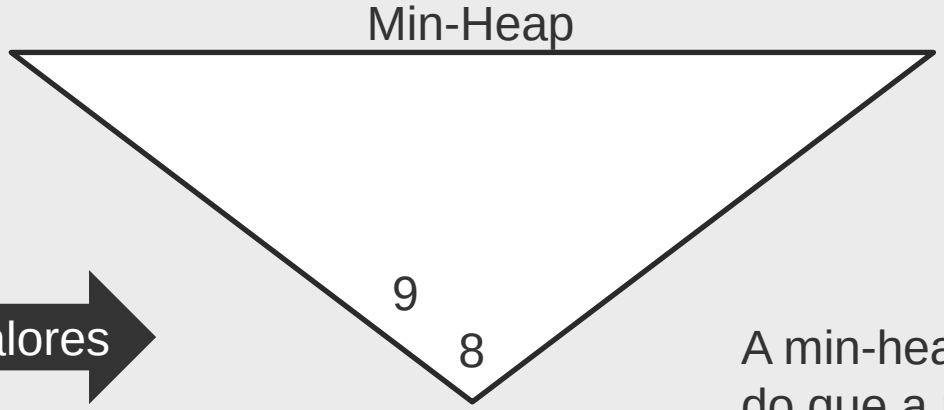


Heap Binária

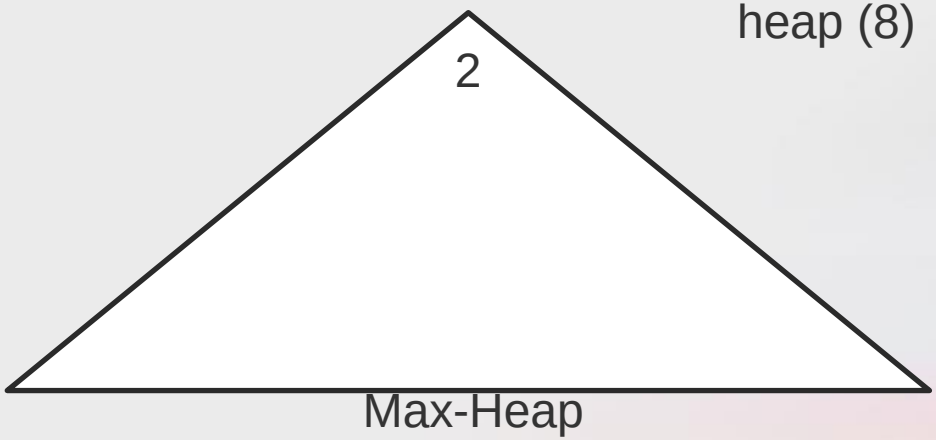
- Exemplo: Cálculo da mediana de uma sequência de valores

Sequencia de valores

… 12 , 4, 1 , 3 , 7



A min-heap tem mais valores do que a max-heap. A mediana é o topo da min-heap (8)

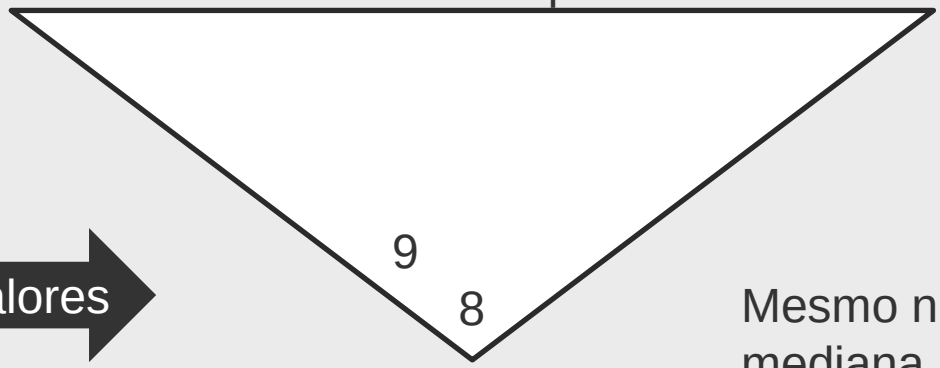




Heap Binária

- Exemplo: Cálculo da mediana de uma sequência de valores

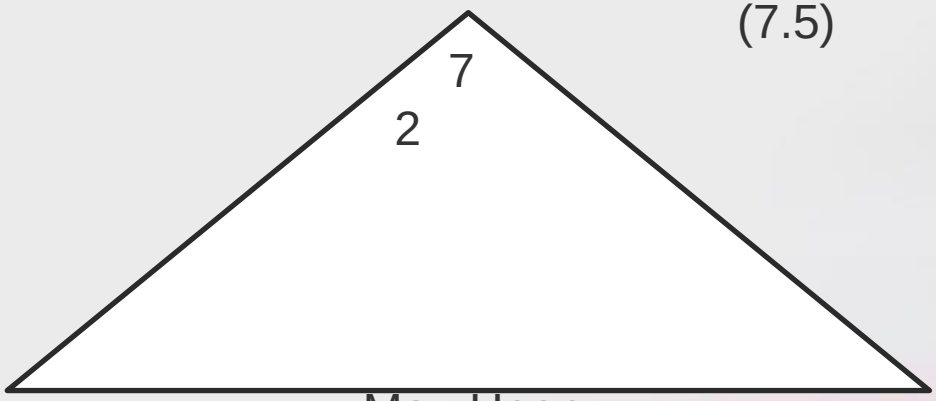
Min-Heap



Sequencia de valores

… 12 , 4, 1 , 3

Mesmo número de valores, a mediana é de novo valor médio de ambos os topos (7.5)



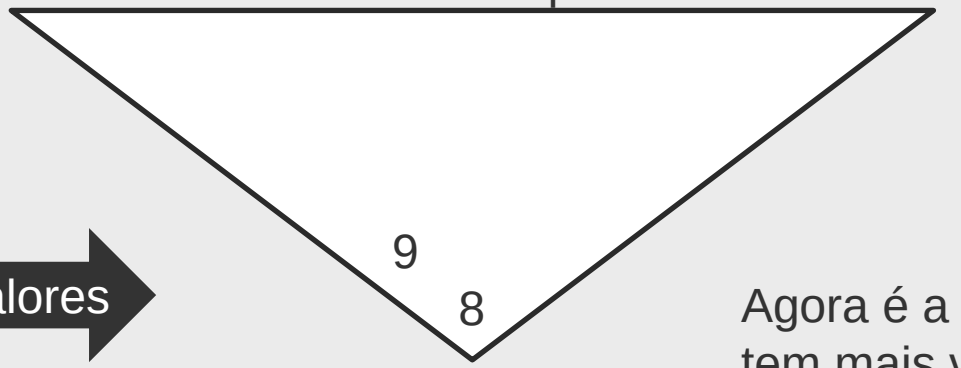
Max-Heap



Heap Binária

- Exemplo: Cálculo da mediana de uma sequência de valores

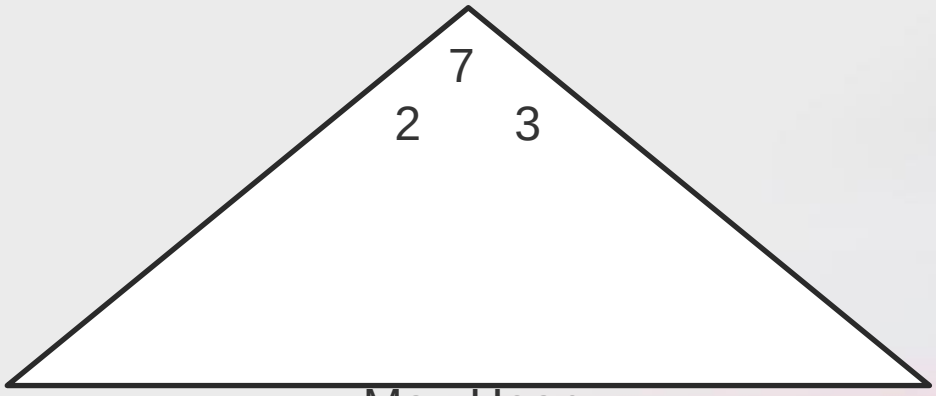
Min-Heap



Sequencia de valores

… 12 , 4, 1

Agora é a Max-Heap que tem mais valores, a mediana é o topo da max-heap. (7)

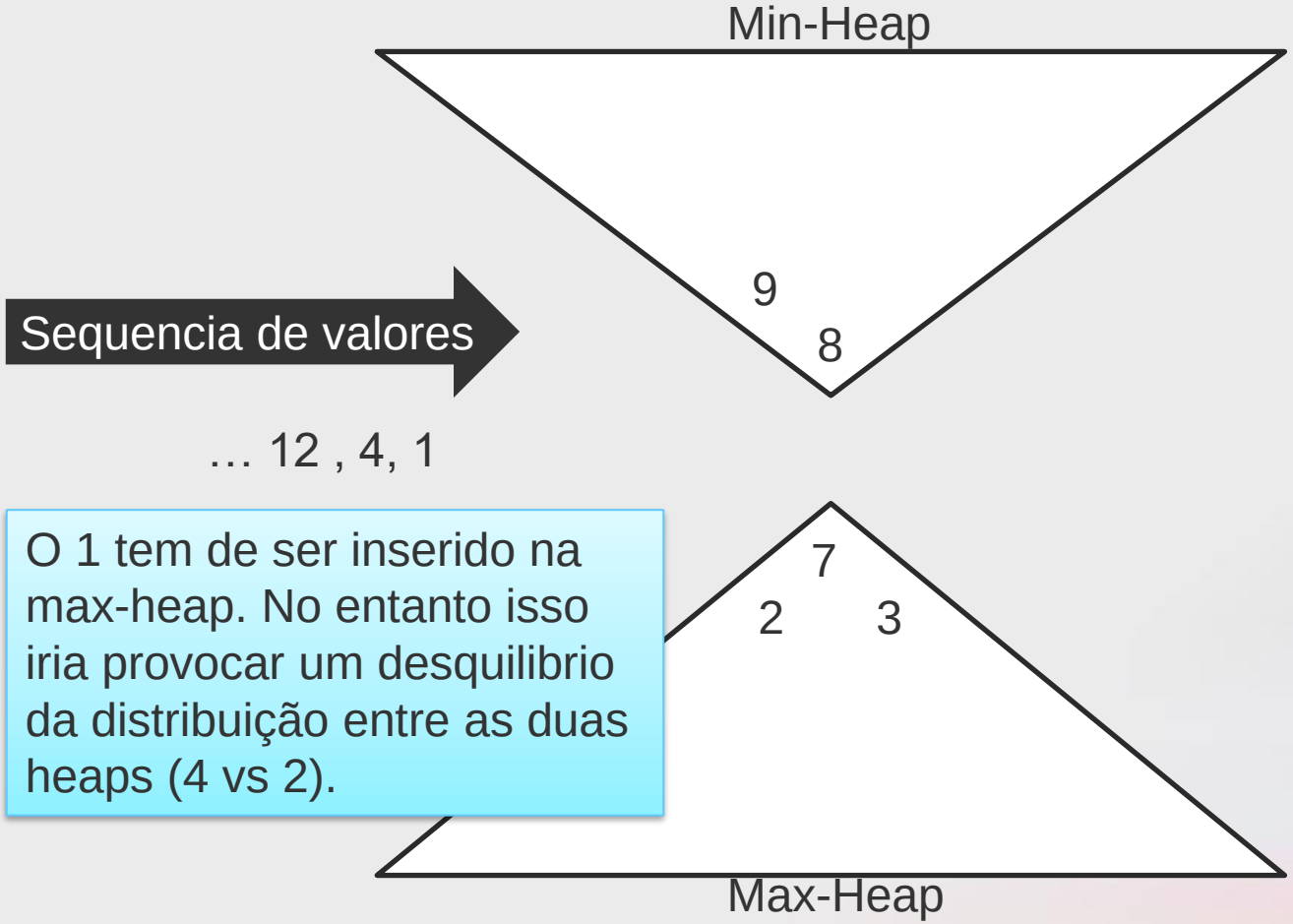


Max-Heap



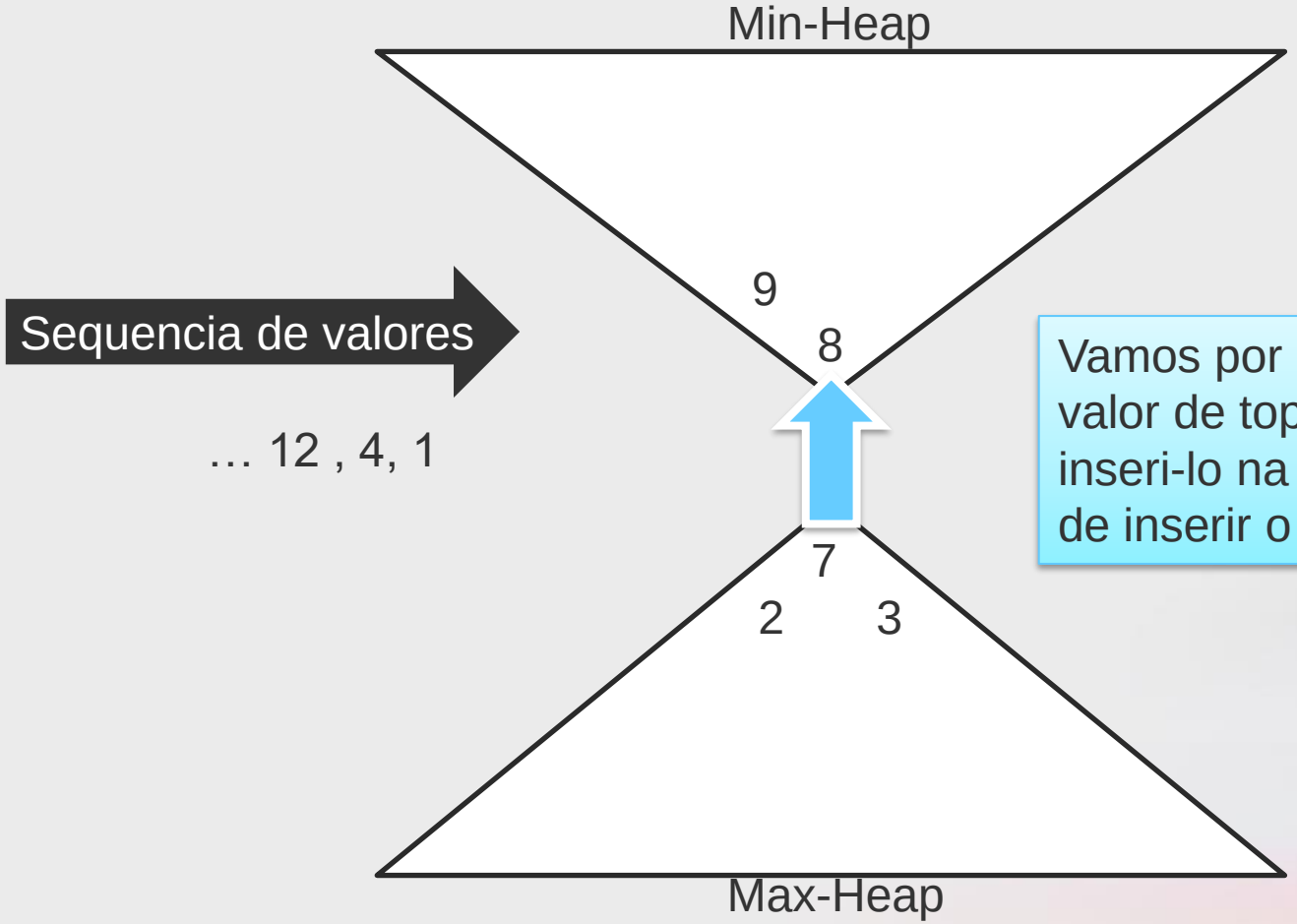
Heap Binária

- Exemplo: Cálculo da mediana de uma sequência de valores



Heap Binária

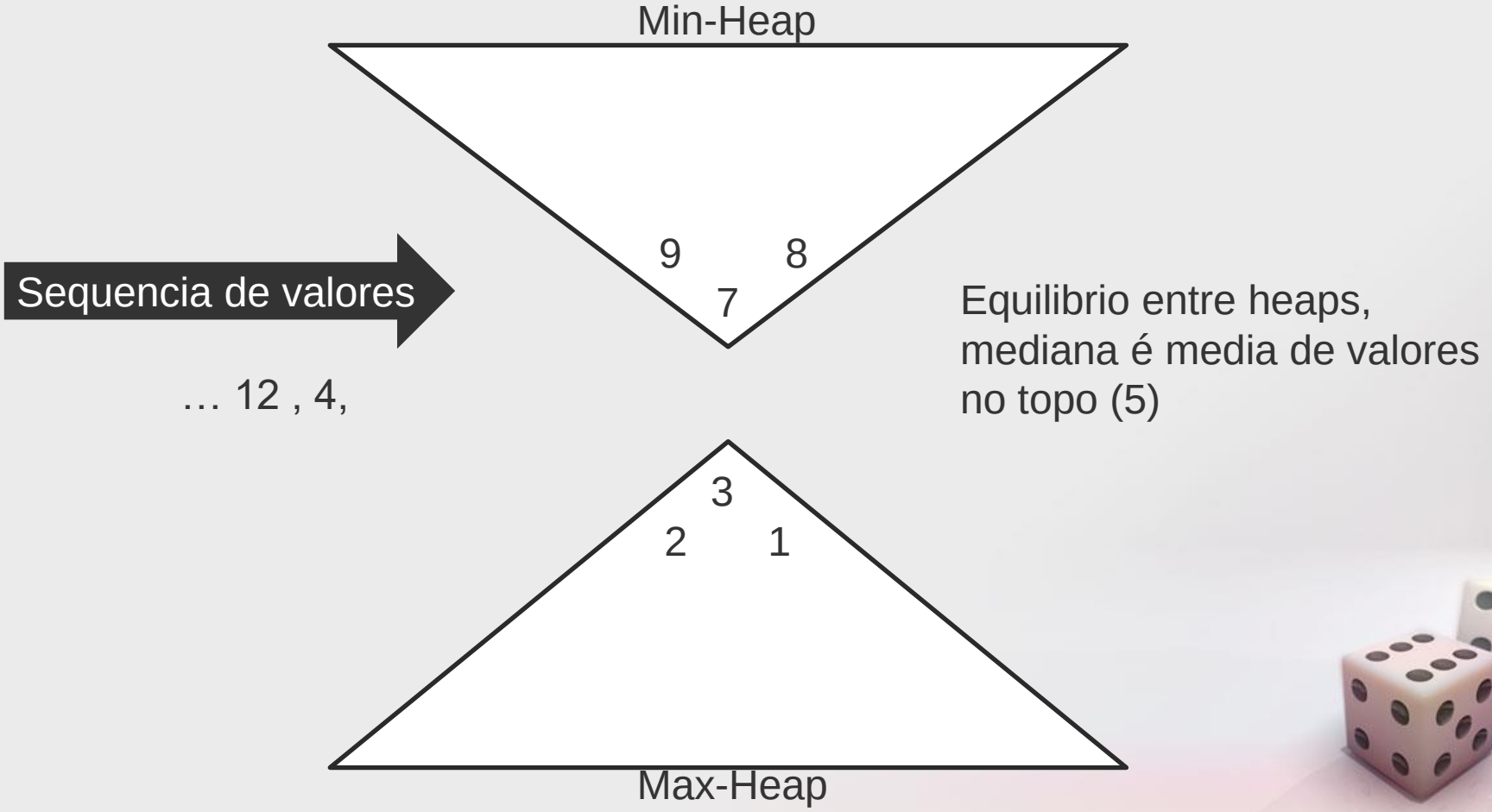
- Exemplo: Cálculo da mediana de uma sequência de valores



Vamos por isso remover o valor de topo da Max-heap e inseri-lo na min-heap, antes de inserir o número 1.

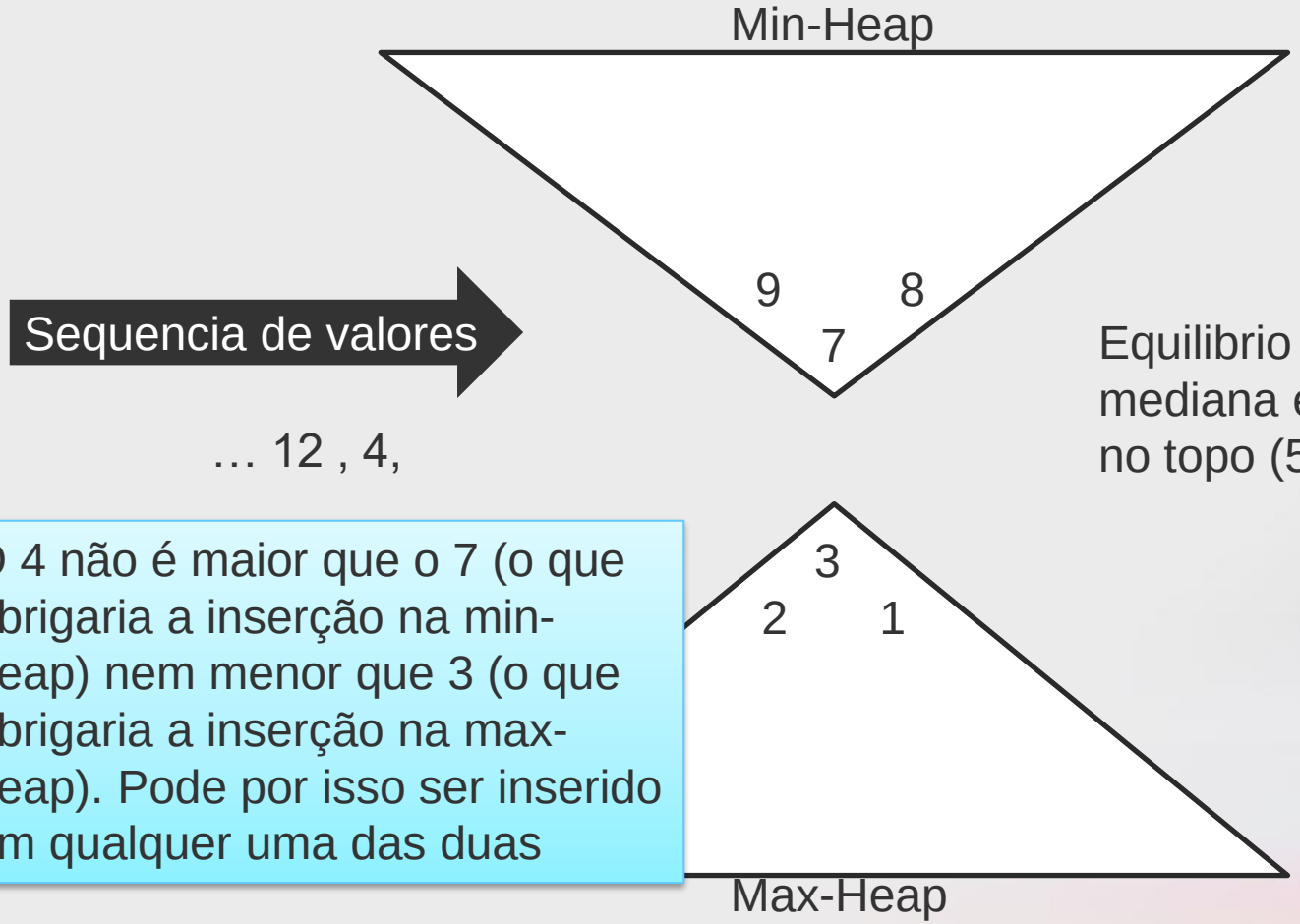
Heap Binária

- Exemplo: Cálculo da mediana de uma sequência de valores



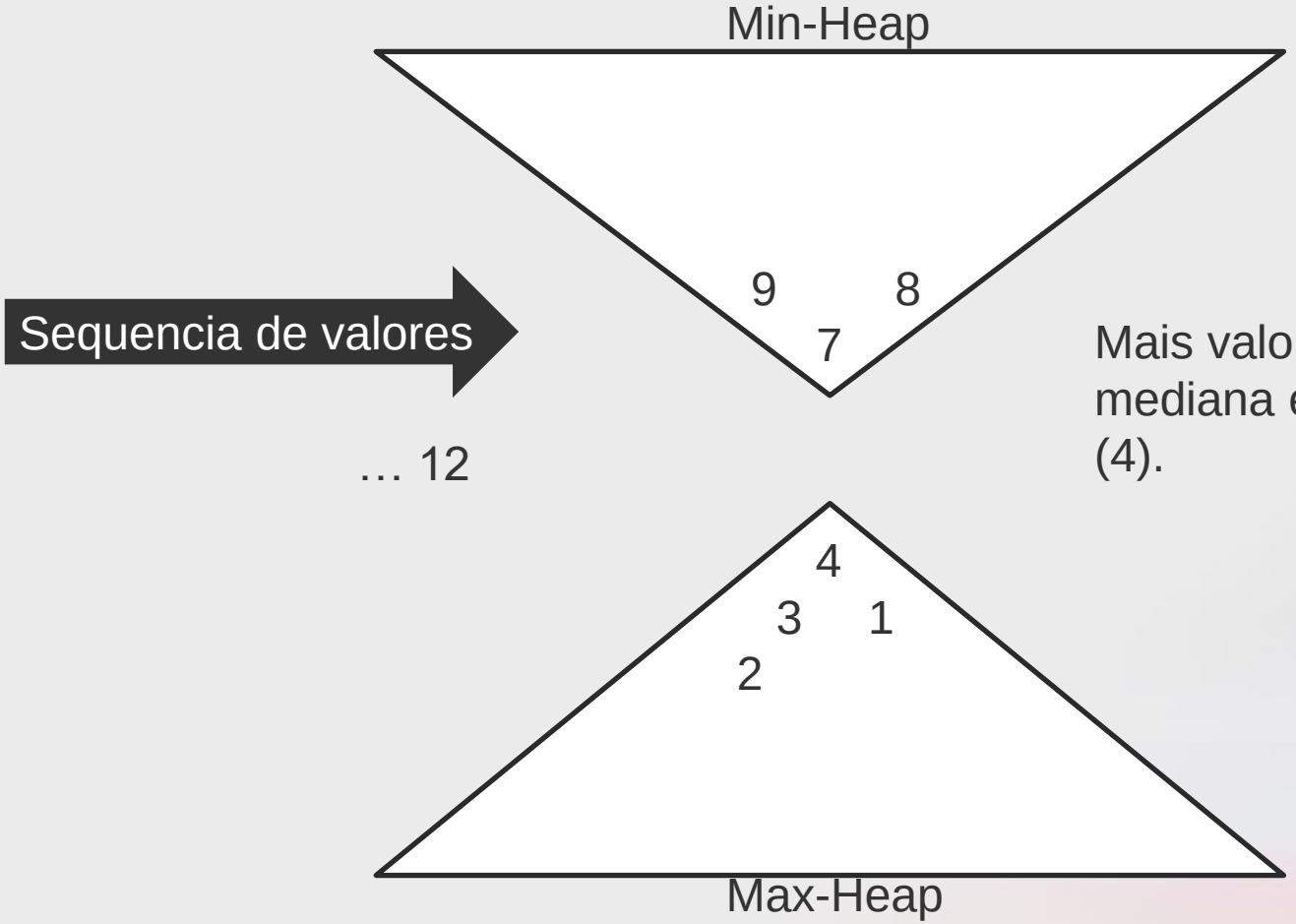
Heap Binária

- Exemplo: Cálculo da mediana de uma sequência de valores



Heap Binária

- Exemplo: Cálculo da mediana de uma sequência de valores



Heap Binária

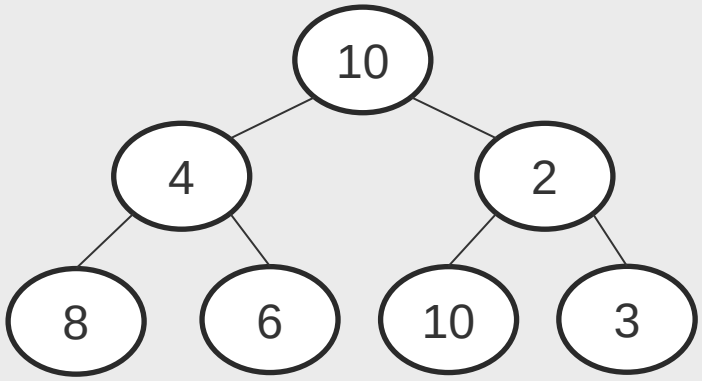
- Exemplo: Ordenação *(Heapsort)*
  - Inicializar a partir do array desordenado uma MAX-HEAP através do processo anteriormente analisado (complexidade O(N))
  - Remover o maior valor (O(log N)). Aproveitar o facto de a maior posição do array se encontrar agora “livre” (porque a heap é uma ABC) para guardar lá o valor removido.
  - Repetir o ponto anterior N vezes. Terminado esse processo, o array encontra-se ordenado, com tempo O(N log N).



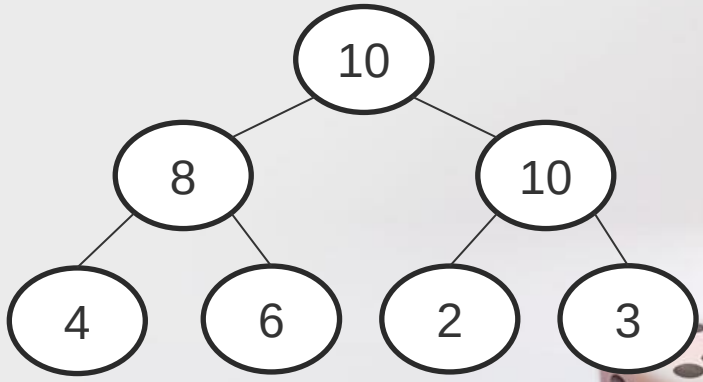
Heap Binária

- Exemplo: Ordenação *(Heapsort)*
  - Inicializar a partir do array desordenado uma MAX-HEAP através do processo anteriormente analisado (complexidade O(N))

10	4	2	8	6	10	3



10	8	10	4	6	2	3



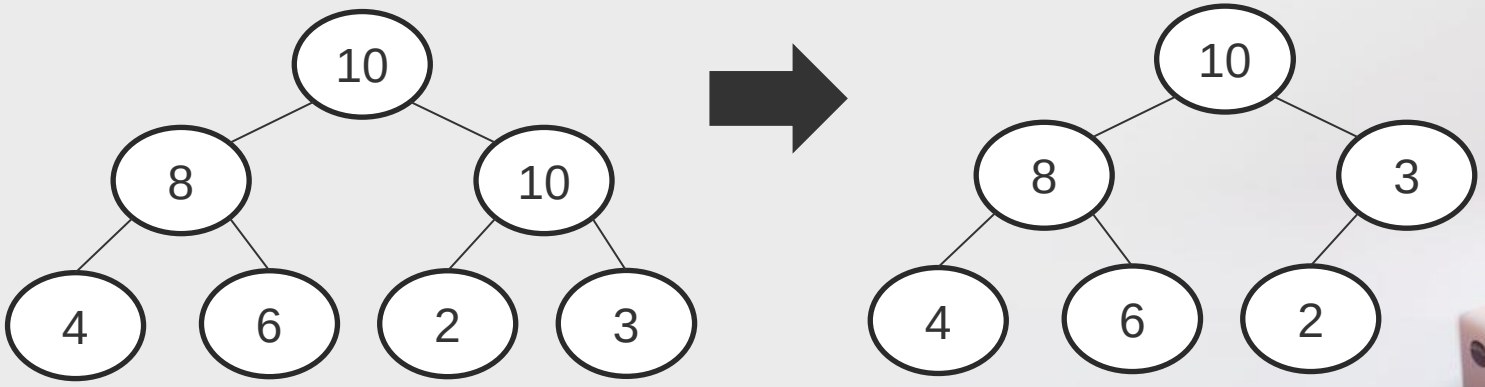


Heap Binária

- Exemplo: Ordenação *(Heapsort)*
  - Remover o maior valor (O(log N)). Aproveitar o facto de a maior posição do array se encontrar agora “livre” (porque a heap é uma ABC) para guardar lá o valor removido.

10	8	10	4	6	2	3

10	8	3	4	6	2	10

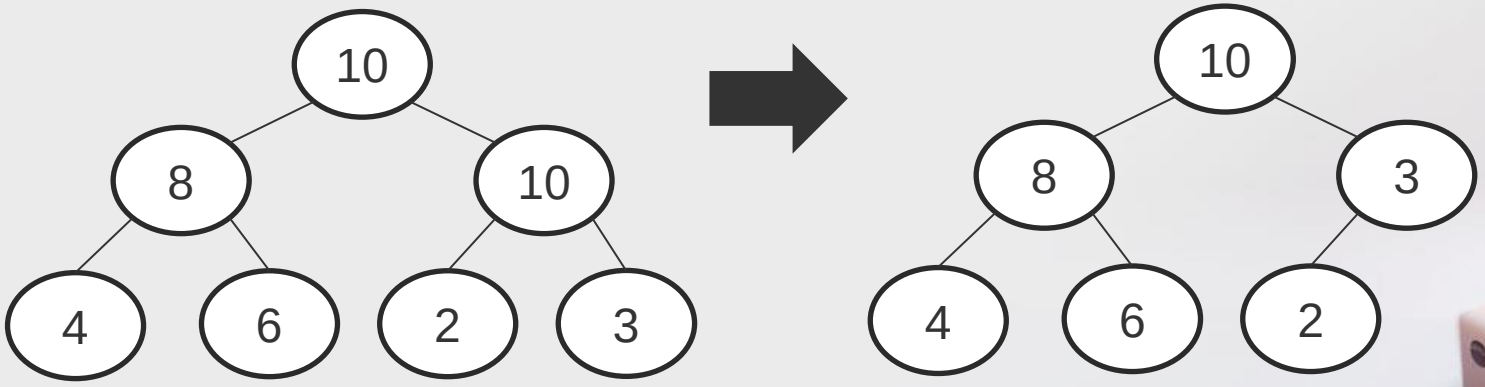


Heap Binária

- Exemplo: Ordenação *(Heapsort)*
  - Remover o maior valor (O(log N)). Aproveitar o facto de a maior posição do array se encontrar agora “livre” (porque a heap é uma ABC) para guardar lá o valor removido.

10	8	10	4	6	2	3

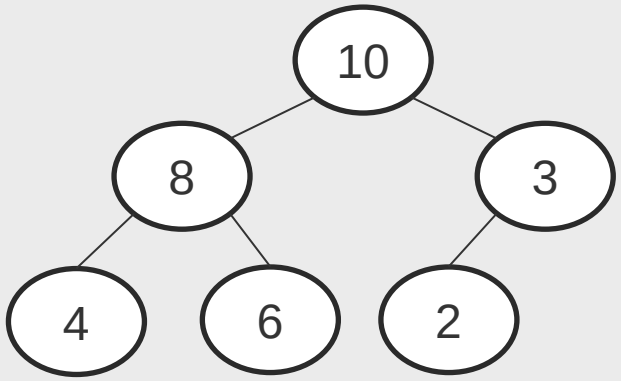
10	8	3	4	6	2	10



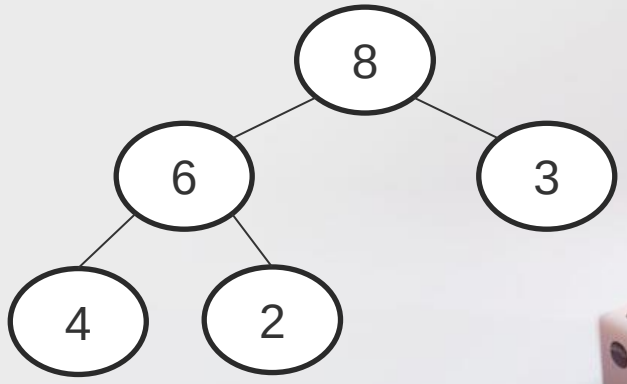
Heap Binária

- Exemplo: Ordenação *(Heapsort)*
  - Repetir o ponto anterior N vezes. Terminado esse processo, o array encontra-se ordenado, com tempo O(N log N).

10	8	3	4	6	2	10



8	6	3	4	2	10	10

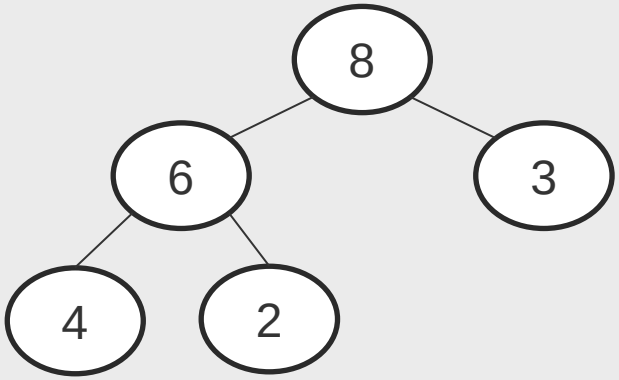




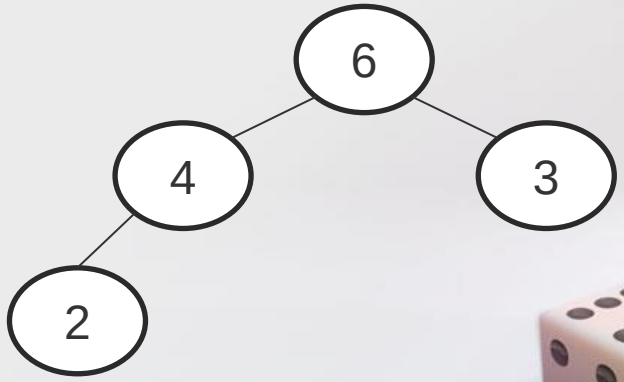
Heap Binária

- Exemplo: Ordenação *(Heapsort)*
  - Repetir o ponto anterior N vezes. Terminado esse processo, o array encontra-se ordenado, com tempo O(N log N).

8	6	3	4	2	10	10



6	4	3	2	8	10	10

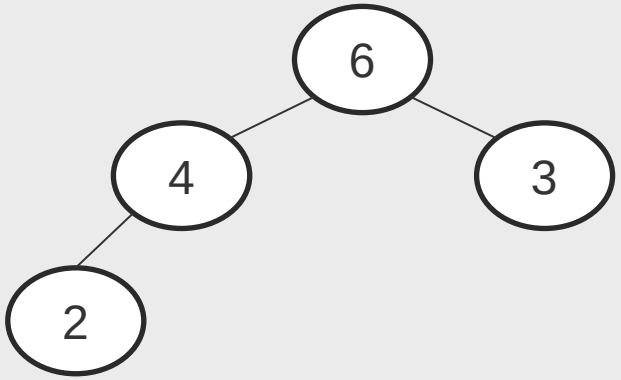


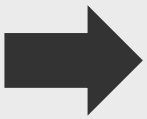


Heap Binária

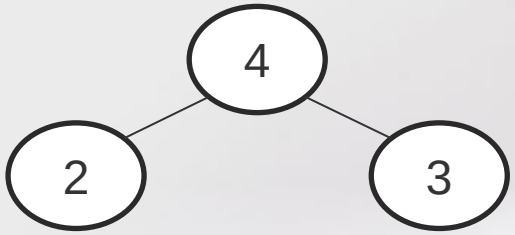
- Exemplo: Ordenação *(Heapsort)*
  - Repetir o ponto anterior N vezes. Terminado esse processo, o array encontra-se ordenado, com tempo O(N log N).

6	4	3	2	8	10	10





4	2	3	6	8	10	10

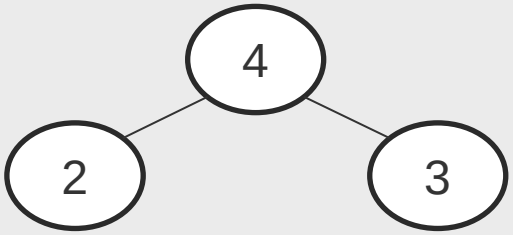


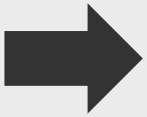


Heap Binária

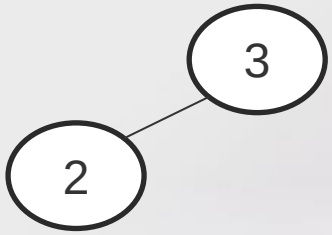
- Exemplo: Ordenação *(Heapsort)*
  - Repetir o ponto anterior N vezes. Terminado esse processo, o array encontra-se ordenado, com tempo O(N log N).

4	2	3	6	8	10	10





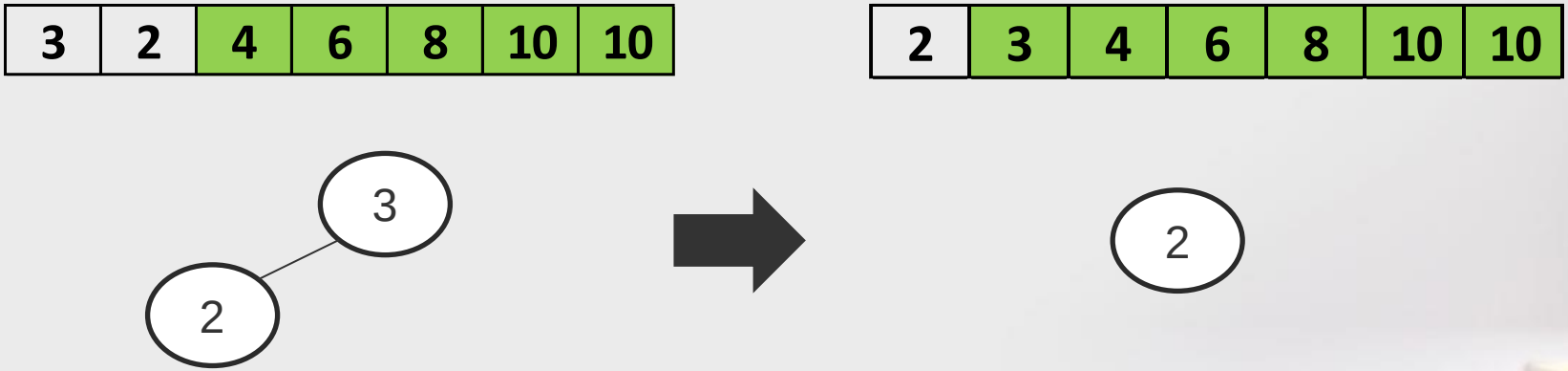
3	2	4	6	8	10	10





Heap Binária

- Exemplo: Ordenação *(Heapsort)*
  - Repetir o ponto anterior N vezes. Terminado esse processo, o array encontra-se ordenado, com tempo O(N log N).



Heap Binária

- Exemplo: Ordenação *(Heapsort)*
  - O array está ordenado. Foram efetuadas:
    - uma operação linear (inicialização)
    - N operações logarítmicas (remoção do maior)
  - Logo, a complexidade total é dada por O(N * Log N).

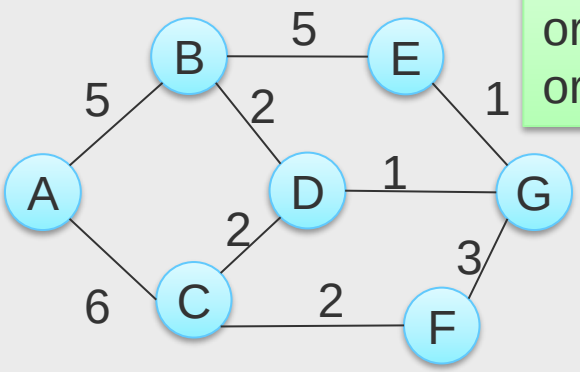
2	3	4	6	8	10	10



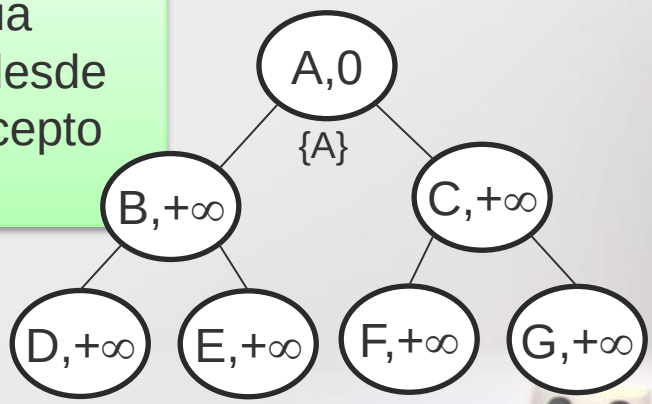
Heap Binária

- Exemplo: Algoritmos de grafos
- *Algoritmo de Dijkstra para encontrar menor distância a partir de um ponto de origem num grafo.*

Distância a partir do nodo A



Heap inicializada com nodos que representam melhor distância conhecida neste momento desde origem (+INF para todos excepto origem com dist=0).

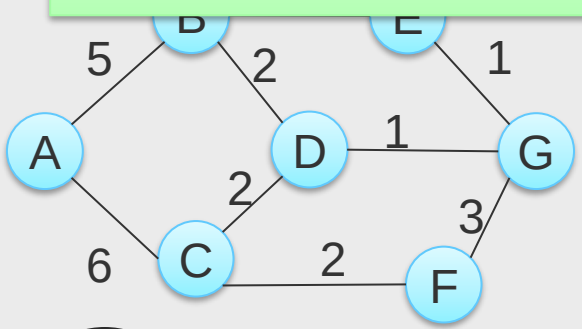


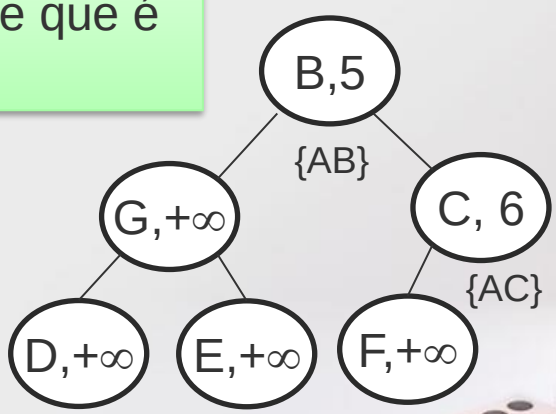
Heap Binária

- Exemplo: Algoritmos de grafos
- *Algoritmo de Dijkstra para encontrar menor distância a partir de um ponto de origem num grafo.*

Distância a partir do nodo A

Remove menor valor da Heap, acrescenta-o a resultado, expande a partir desse nó para vizinhos, possivelmente encontrado um caminho mais curto do que aquele que é atualmente conhecido. Atualiza a heap





A,0

{A}

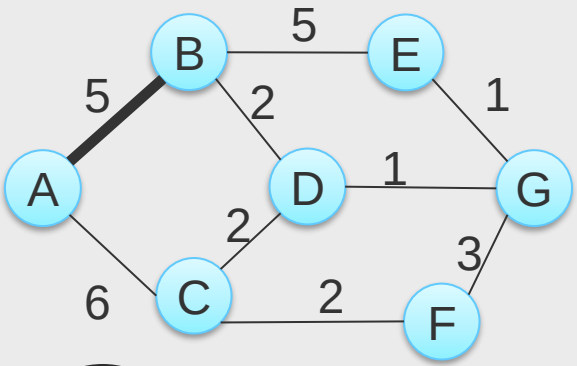
Resultado (distância a A é 0, percurso {A})



Heap Binária

- Exemplo: Algoritmos de grafos
- *Algoritmo de Dijkstra para encontrar menor distância a partir de um ponto de origem num grafo.*

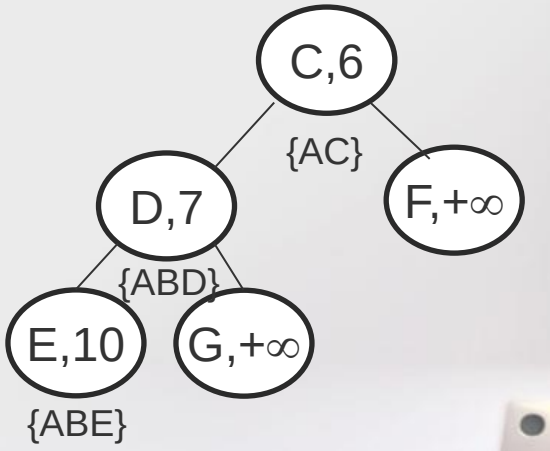
Distância a partir do nodo A



A,0 {A}

B,5 {AB}

Retira (B,5) e acrescenta a resultado, atualiza conjunto de vizinhos de B (que ainda não estão em resultado), e atualiza heap.
Repete até completo.

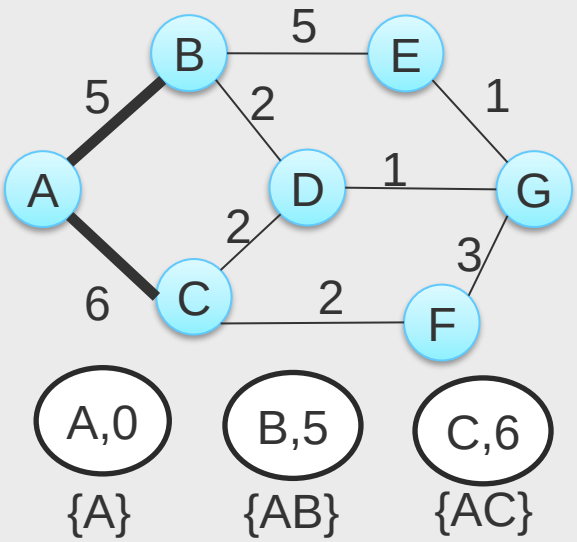




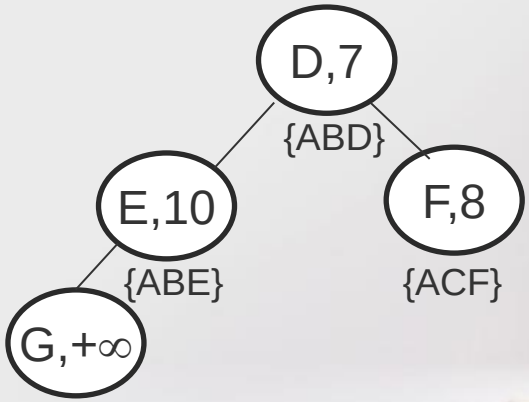
Heap Binária

- Exemplo: Algoritmos de grafos
- *Algoritmo de Dijkstra para encontrar menor distância a partir de um ponto de origem num grafo.*

Distância a partir do nodo A



Etc, …

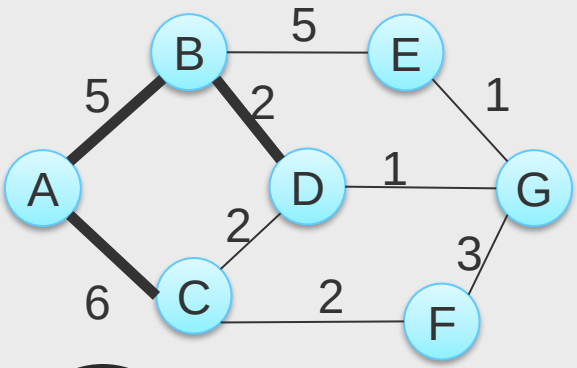


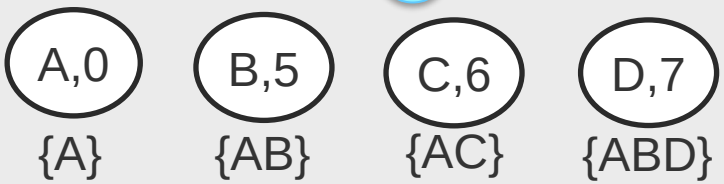


Heap Binária

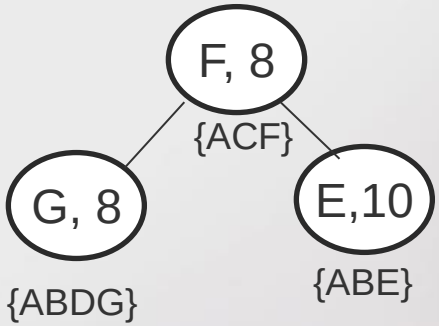
- Exemplo: Algoritmos de grafos
- *Algoritmo de Dijkstra para encontrar menor distância a partir de um ponto de origem num grafo.*

Distância a partir do nodo A





Etc, …

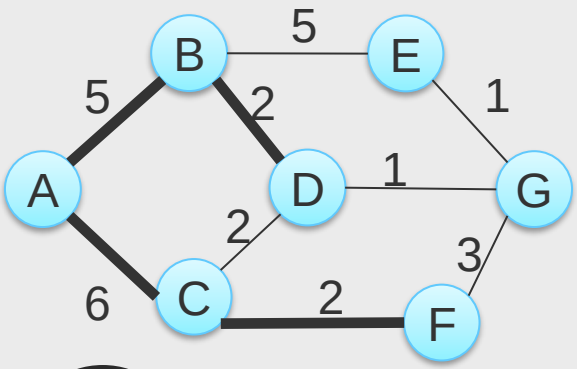




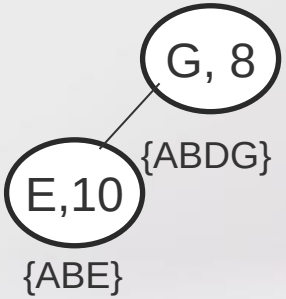
Heap Binária

- Exemplo: Algoritmos de grafos
- *Algoritmo de Dijkstra para encontrar menor distância a partir de um ponto de origem num grafo.*

Distância a partir do nodo A



Etc…



A,0 {A}

B,5 {AB}

C,6 {AC}

D,7 {ABD}

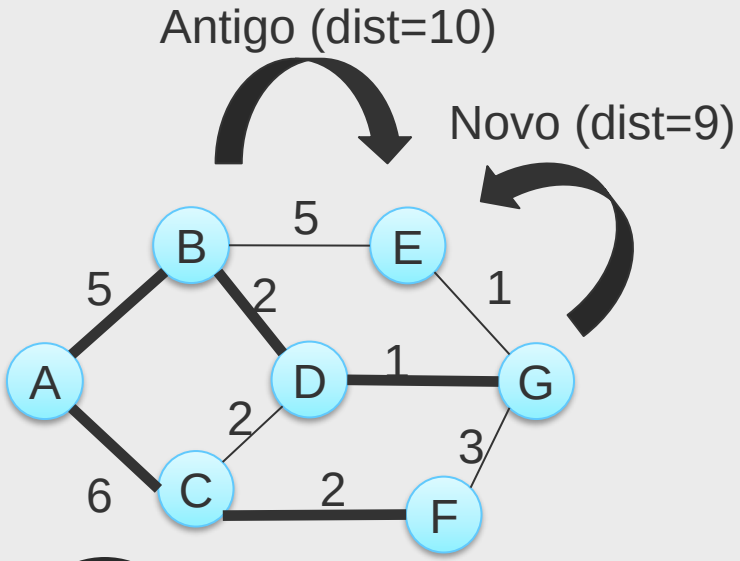
F, 8 {ACF}



Heap Binária

- Exemplo: Algoritmos de grafos
- *Algoritmo de Dijkstra para encontrar menor distância a partir de um ponto de origem num grafo.*

Distância a partir do nodo A



A distância até ao nodo E é reduzida de 10 para 9, dado que o novo caminho {ABDGE} é mais curto do que o anterior {ABE}

E,9
{ABDGE}

A,0 {A}

B,5 {AB}

C,6 {AC}

D,7 {ABD}

F, 8 {ACF}

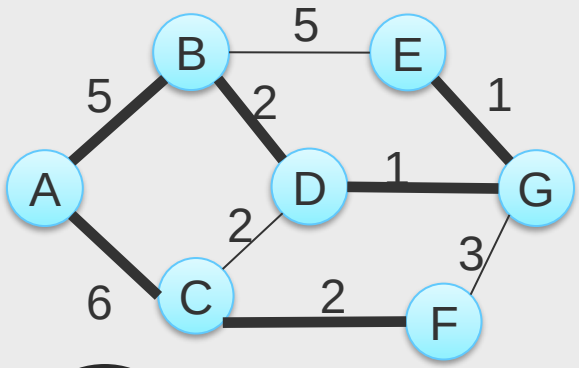
G, 8 {ABDG}



Heap Binária

- Exemplo: Algoritmos de grafos
- *Algoritmo de Dijkstra para encontrar menor distância a partir de um ponto de origem num grafo.*

Distância a partir do nodo A



Terminámos. A heap está vazia. Foram encontrados os caminhos com menor distância de A até todos os outros nós.

A,0	B,5	C,6	D,7	F, 8	G, 8	E,9
{A}	{AB}	{AC}	{ABD}	{ACF}	{ABDG}	{ABDGE}

